Exam Author

Premium exam material

Get certification quickly with the Exam Author Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.
https://www.ExamAuthor.com

Anthropic

(Claude Certified Architect - Foundations)

Claude Certified Architect – Foundations (CCA-F) Certification

Total: **85 Questions**

Link: https://examauthor.com/papers/anthropic/claude-certified-architect

Question: 1 **Exam Author**

After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot complete the task because no research findings were provided. What is the most likely cause of this issue?

A. The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.

B. The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.

C. The subagents need to share a single API connection to enable automatic context sharing between invocations.

D. The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Answer: D

Explanation:

A. The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories. Incorrect.

Agents do not require direct access to each other's histories. Proper orchestration passes outputs explicitly via prompts.

B. The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents. Incorrect.

If this were the issue, the agent would receive truncated data, not no data at all. The error indicates missing inputs entirely.

C. The subagents need to share a single API connection to enable automatic context sharing between invocations. Incorrect.

Agent communication does not depend on shared API connections. Context must be explicitly passed by the coordinator.

D. The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt. Correct.

The synthesis agent can only act on the information provided in its prompt. If prior outputs are not passed, it will report missing research findings.

Question: 2 **Exam Author**

When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2022: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

A. Require subagents to include publication or data collection dates in their structured outputs.

B. Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.

C. Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.

D. Configure the web search agent to only return results from the past 6 months

Answer: A

Explanation:

A. Require subagents to include publication or data collection dates in their structured outputs. Correct.

Providing timestamps allows the synthesis agent to understand that the figures refer to different points in time, enabling it to interpret the data as a trend (growth) rather than a contradiction.

B. Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix. Incorrect.

This approach hides useful context and does not help the agent understand relationships between data points over time.

C. Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric. Incorrect.

Discarding older data removes valuable historical insight and prevents trend analysis.

D. Configure the web search agent to only return results from the past 6 months. Incorrect.

Limiting recency reduces context and does not address the core issue of interpreting time-based differences.

Question: 3 **Exam Author**

Users report that final reports sometimes lack depth on specific subtopics. Investigation shows that the document analysis agent frequently identifies gaps — for instance, noting "the retrieved sources discuss API authentication but lack details on token refresh patterns" — but under the current strict pipeline, this insight isn't actionable since search has already completed. What is the most effective architectural change?

A. Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient.

B. Add a research planning agent before the search phase that decomposes topics into specific sub-questions.

C. Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review.

D. Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected.

Answer: A

Explanation:

A. Have the analysis agent report specific gaps to the coordinator, which triggers targeted searches and re-invokes analysis until sufficient. Incorrect.

While effective in theory, this tightly couples the analysis agent with orchestration logic. Agents should not control workflow decisions directly.

B. Add a research planning agent before the search phase that decomposes topics into specific sub-questions. Incorrect.

Planning helps upfront coverage, but it does not solve the issue of new gaps discovered during analysis.

C. Have the synthesis agent attach confidence scores to each section and flag areas with insufficient coverage for manual review. Incorrect.

This only identifies the problem but does not automatically resolve missing depth.

D. Have the coordinator review analysis output for gap indicators and re-invoke search with gap-informed queries when gaps are detected. Correct.

This keeps orchestration centralized and enables an iterative loop where newly discovered gaps trigger additional targeted searches and analysis, leading to more complete results.

Question: 4 **Exam Author**

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially complete and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach be Information fidelity with context efficiency when restoring agent state?

A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.

B. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

C. Have each agent maintain its own persistent state file and reload it independently at the start of each session.

D. Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings.

Answer: A

Explanation:

A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts. Correct.

This provides **high information fidelity** (structured, complete outputs) while maintaining **context efficiency** (only relevant pieces are re-injected into prompts). The coordinator remains in control of what each agent needs, avoiding unnecessary bloat and duplication.

B. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming. Incorrect.

Conversation logs are often verbose and unstructured, leading to **context overload** and inefficient prompt usage without guaranteed clarity.

C. Have each agent maintain its own persistent state file and reload it independently at the start of each session. Incorrect.

This decentralizes control and can lead to **inconsistencies and coordination issues**, especially when agents need shared or aligned context.

D. Index all agent outputs in a shared vector store. When resuming each agent queries the store using semantic search to retrieve relevant prior findings. Incorrect.

Vector stores are useful for retrieval, but they introduce **probabilistic recall** and may miss or distort critical structured state, reducing fidelity during recovery.

Question: 5 **Exam Author**

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage. What change would most effectively improve research completeness?

A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before Invoking synthesis again.

B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

C. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.

D. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.

Answer: A

Explanation:

A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again. Correct.

This introduces an **iterative feedback loop**, where identified gaps are actively addressed. The coordinator maintains control and ensures completeness before final report generation.

B. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information. Incorrect.

Broader queries may help coverage but are inefficient and still won't guarantee that **specific gaps discovered later** are filled.

C. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output. Incorrect.

This improves transparency but does **not solve the completeness problem**.

D. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator. Incorrect.

This breaks **separation of concerns** and reduces system control. The coordinator should manage task delegation, not the synthesis agent.

Question: 6 **Exam Author**

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

A. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations

B. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents

C. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reading single-precedent granularity

D. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis

Answer: D

Explanation:

A. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations. Incorrect.

This decentralizes orchestration and makes the system harder to monitor and debug. The coordinator loses visibility into dynamically spawned agents.

B. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents. Incorrect.

While this improves scalability, it introduces infrastructure complexity and reduces transparency for debugging at the coordinator level.

C. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity. Incorrect.

This further complicates the architecture and makes tracing execution paths difficult, reducing observability and control.

D. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis. Correct.

This enables **parallel processing** to reduce latency while keeping orchestration centralized. The coordinator retains full visibility, making monitoring and debugging easier.

Question: 7 **Exam Author**

Introduction monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent - neither requires the other's output. How should you modify the system to run these subagents concurrently?

A. Switch both subagents to use a Haiku tier model instead of to reduce their individual execution time.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator subagent pair, then aggregates results.

C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same

D. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.

Answer: D

Explanation:

A. Switch both subagents to use a Haiku tier model instead to reduce their individual execution time. Incorrect.

This may reduce latency per task, but it does **not address the core issue** of sequential execution vs. parallelism.

B. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator subagent pair, then aggregates results. Incorrect.

This overcomplicates the architecture and duplicates coordinators unnecessarily instead of fixing

concurrency within the existing flow.

C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time. Incorrect.

Instructions alone are not reliable for enforcing concurrency. Execution behavior depends on how tool calls are structured, not just prompt wording.

D. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns. Correct.

Issuing both tool calls in one response enables **true parallel execution**, since the system can run them concurrently instead of waiting for one to finish before starting the other.

Question: 8 **Exam Author**

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing other times reports over hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "Industry analysts estimate $50B market size (methodolo the document analysis agent returns "peer-reviewed study estimates $358 (1578, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the ma 6358-6508 depending on factors." What systematic approach best addresses this?

A. Configure subagents to only report findings meeting a high confidence threshold, filtering uncertain information before it reaches the coordinator.

B. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

C. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context.

D. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calculated reliability scores to produce a statistically grounded synthesis.

Answer: C

Explanation:

A. Configure subagents to only report findings meeting a high confidence threshold, filtering uncertain information before it reaches the coordinator. Incorrect.

This suppresses potentially valuable but uncertain insights and introduces bias by hiding ambiguity rather than managing it.

B. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources. Incorrect.

While useful for validation, this approach still filters out uncertainty instead of representing it, and may discard novel or emerging insights.

C. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterization and methodological context. Correct.

This directly addresses inconsistent handling of uncertainty by **making it explicit and structured**, allowing users to understand both consensus and disagreement without losing context.

D. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to

standardized probability scores (0.0–1.0), then weight-average findings by their calculated reliability scores to produce a statistically grounded synthesis. Incorrect.

This introduces artificial precision and may oversimplify complex, qualitative uncertainty, potentially misleading users.

Question: 9 **Exam Author**

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds. While this might be acceptable for complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

A. Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.

B. Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

C. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

D. Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.

Answer: C

Explanation:

A. Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination. Incorrect.

This is rigid and brittle. As query patterns evolve, maintaining rules becomes difficult and coverage gaps are likely.

B. Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve. Incorrect.

While adaptive, this introduces **model maintenance overhead**, requires labeled data, and may lag behind new or rare query types.

C. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements. Correct.

This provides **flexible, real-time routing** without rigid rules or heavy ML infrastructure. The coordinator can tailor execution paths to query complexity efficiently.

D. Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness. Incorrect.

This is overly simplistic and risks misclassification, reducing accuracy or missing nuance for queries that appear simple but require deeper analysis.

Question: 10 **Exam Author**

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured

lists of technology synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summari narrative flow. What change would most improve briefing quality?

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technical lists as structured points.

B. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation (such as Markdown) to facilitate more flexible rendering.

C. Standardize all subagent outputs to JSON with fields for every data type to ensure programmatic consistency across the pipeline.

D. Standardize all subagent outputs to prose summaries with a uniform character to maintain a consistent executive voice regardless of the source material.

Answer: A

Explanation:

A. Update the synthesis agent to render each content type appropriately — financial data as tables, news as prose, and technical lists as structured points. Correct.

This preserves the **natural structure and strengths of each data type**, improving clarity, readability, and usefulness of the final briefing.

B. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation (such as Markdown) to facilitate more flexible rendering. Incorrect.

While helpful for consistency, this does not guarantee **appropriate presentation** of different content types and may still lead to generic formatting.

C. Standardize all subagent outputs to JSON with fields for every data type to ensure programmatic consistency across the pipeline. Incorrect.

This improves structure but shifts complexity to the synthesis stage and does not inherently improve **human-readable output quality**.

D. Standardize all subagent outputs to prose summaries with a uniform character to maintain a consistent executive voice regardless of the source material. Incorrect.

This sacrifices important structure (like tables and lists), reducing clarity and effectiveness for data-heavy content.

Question: 11 **Exam Author**

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate — it generates messages like "I'll ask the web search agent to find sources on this topic" — but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

A. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in Claude AgentOptions.

B. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

C. The coordinator's allowed Tools configuration doesn't include "Task", so while it can reason about delegation, cannot invoke the tool required to spawn subagents.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the

available subagent types, preventing the model from knowing they can be invoked.

Answer: C

Explanation:

A. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in Claude AgentOptions. Incorrect.

Even with context isolation, subagents would still be invoked — the issue here is that **no invocation happens at all**, not that context is missing.

B. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified. Incorrect.

Token limits might truncate responses, but this would typically produce malformed outputs or errors — not silent absence of any tool calls.

C. The coordinator's allowed Tools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents. Correct.

The coordinator can **plan and describe delegation**, but without the **Task tool enabled**, it cannot actually execute subagent calls — resulting in no errors but no execution.

D. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked. Incorrect.

While listing agents can help, the model already demonstrates awareness ("I'll ask the web search agent..."). The problem is **execution capability**, not awareness.

Question: 12 **Exam Author**

In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

A. Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

B. Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations.

C. Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

D. Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

Answer: D

Explanation:

A. Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents. Incorrect.

This relies on **post-hoc inference**, which is error-prone and can misattribute claims due to semantic ambiguity.

B. Have the coordinator inject source identifier prefixes into text before each handoff, then parse these prefixes at report generation to reconstruct citations. Incorrect.

This is a fragile, text-based workaround that can break during transformations and doesn't scale well.

C. Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation. Incorrect.

This adds unnecessary complexity and still depends on indirect reconstruction rather than preserving attribution explicitly.

D. Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources. Correct.

This ensures **end-to-end attribution fidelity** by keeping claim-to-source relationships explicit and structured throughout the pipeline, preventing loss during synthesis.

Question: 13 **Exam Author**

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings. What is the correct approach for providing the synthesis subagent with the information it needs?

A. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks

B. Include the complete findings from both subagents directly in the synthesis subagent's prompt

C. Pass reference Identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results

D. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator

Answer: C

Explanation:

A. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks. Incorrect.

This introduces unnecessary coupling and complexity. Subagents shouldn't need to actively fetch data from others.

B. Include the complete findings from both subagents directly in the synthesis subagent's prompt. Incorrect.

While simple, this approach does **not scale well** for large outputs and can exceed context limits, reducing efficiency.

C. Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results. Correct.

This is the **most scalable and production-ready approach**. It preserves **information fidelity** while avoiding context bloat, allowing the synthesis agent to retrieve exactly what it needs.

D. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator. Incorrect.

There is no automatic context inheritance — without explicit data access, the synthesis agent cannot function properly.

Question: 14 **Exam Author**

The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information flow between these two specialized subagents?

A. "The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

B. The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.

C. The web search agent directly invokes the document analysis agent, using the discovered sources as parameters.

D. Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.

Answer: A

Explanation:

A. The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent. Correct.

This follows the standard orchestration pattern where the **coordinator manages all data flow**, explicitly passing outputs between subagents.

B. The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events. Incorrect.

This introduces unnecessary infrastructure complexity and is not the typical agent orchestration model.

C. The web search agent directly invokes the document analysis agent, using the discovered sources as parameters. Incorrect.

Subagents should not invoke each other directly; this breaks **centralized control and observability**.

D. Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them. Incorrect.

While possible in advanced systems, this is not the standard or simplest approach; it adds complexity without clear necessity in typical pipelines.

Question: 15 **Exam Author**

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

A. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.

B. Pass the full accumulated context from all prior agents.

C. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and ant Irant excerpts.

D. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.

Answer: C

Explanation:

A. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated. Incorrect.

This relies on **post-hoc reconstruction**, which is error-prone and can lead to incorrect or missing citations.

B. Pass the full accumulated context from all prior agents. Incorrect.

This ensures completeness but is highly inefficient (120K+ tokens) and risks exceeding context limits.

C. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts. Correct.

This provides the **best balance of completeness and efficiency** — retaining precise attribution while keeping context size manageable.

D. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only. Incorrect.

This loses granularity and makes precise citation mapping difficult, reducing attribution fidelity.

Question: 16 **Exam Author**

Your search products tool queries an external catalog API that returns paginated results (50 items per request). Production logs show queries frequently match 200+ products, and the design that auto-fetches all pages causes 15-20 second delays. How should you redesign the pagination handling?

A. Create separate search products and fetch more results tools for pagination.
B. Implement server-side relevance ranking and return only the top 50 most relevant items.
C. Add a max pages parameter (default: 2) that controls how many pages are fetched internally.
D. Return the first page with total match count and cursor for additional pages.

Answer: D

Explanation:

A. Create separate search products and fetch more results tools for pagination. Incorrect.

This exposes pagination mechanics to the agent, increasing complexity and coupling tool usage with control flow.

B. Implement server-side relevance ranking and return only the top 50 most relevant items. Incorrect.

While this reduces latency, it **removes access to the full result set**, limiting flexibility when more results are actually needed.

C. Add a max pages parameter (default: 2) that controls how many pages are fetched internally. Incorrect.

This is an improvement over fetching everything, but it still **hides pagination control inside the tool** and may fetch unnecessary data.

D. Return the first page with total match count and cursor for additional pages. Correct.

This enables **lazy loading and explicit control**, allowing the agent to fetch more results only when needed — balancing performance and completeness.

Question: 17 Exam Author

Your search Flights tool calls an external airline API that occasionally returns a 503 Service Unavailable error. What is the most effective way to handle this error in your tool implementation?

A. Return an empty flight list as if the search succeeded but found no matching flights.

B. Log the error internally and return an empty response, letting the model continue without the flight data.

C. Return an error message in the tool result explaining the service is temporarily unavailable.

D. Automatically retry the request up to five times with exponential backoff before returning results to the agent.

Answer: D

Explanation:

A. Return an empty flight list as if the search succeeded but found no matching flights. Incorrect.

This hides the failure and misleads the system into thinking no flights exist, which can lead to incorrect conclusions.

B. Log the error internally and return an empty response, letting the model continue without the flight data. Incorrect.

This still suppresses the failure signal, preventing the agent from taking corrective action.

C. Return an error message in the tool result explaining the service is temporarily unavailable. Incorrect.

While transparent, this alone doesn't attempt recovery and may degrade user experience unnecessarily.

D. Automatically retry the request up to five times with exponential backoff before returning results to the agent. Correct.

This is the most effective approach—handles **transient failures gracefully**, improves reliability, and only surfaces errors if retries fail.

Question: 18 Exam Author

Your MCP server implements a check_availability tool that queries an external calendar API. During testing, you encounter three error conditions:(1) the tool is called with a malformed request, missing the required user_email parameter(2) the calendar API returns a 404 because the specified user doesn't exist in the calendar system(3) the calendar API returns a 503 because the service is temporarily unavailable.How should each error be reported according to MCP's error handling design?

A. Report all three as tool results with isError: true

B. Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as a tool result with isError: true

C. Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with isError: true

D. Report all three as JSON-RPC protocol errors.

Answer: C

Explanation:

A. Report all three as tool results with isError: true Incorrect.

Malformed requests (error 1) are **protocol-level issues**, not tool execution results, so they should not be reported this way.

B. Report errors 1 and 2 as JSON-RPC protocol errors, report error 3 as a tool result with isError: true
Incorrect.

A 404 (error 2) is a **valid tool execution outcome** (the user doesn't exist), not a protocol error.

C. Report error 1 as a JSON-RPC protocol error, report errors 2 and 3 as tool results with isError: true
Correct.

Error 1 (malformed request) → **JSON-RPC protocol error** (invalid input)
Error 2 (user not found) → **Tool result with isError: true** (valid execution, meaningful failure)
Error 3 (service unavailable) → **Tool result with isError: true** (transient external failure)

D. Report all three as JSON-RPC protocol errors. Incorrect.

Only malformed requests should be protocol errors; external API responses are **tool-level outcomes**, not protocol failures.

Question: 19 **Exam Author**

Your documents (query) tool returns results as "Found 3 documents: Q2 Budget Proposal, Q2 Budget Forecast, Annual Review". You want the agent to document (4, multi) and doc (24, multi). What return format would best enable these multi-step workflows?

A. URLs that users can click to open the document in their browser.
B. Structured data containing document IDs and metadata for each result.
C. A JSON array of document titles extracted from the search results.
D. More detailed human-readable descriptions including the size and authors.

Answer: B

Explanation:

A. URLs that users can click to open the document in their browser. Incorrect.

URLs are useful for users, but not ideal for agents performing multi-step workflows that require **reliable referencing and further operations**.

B. Structured data containing document IDs and metadata for each result. Correct.

This enables the agent to **programmatically reference specific documents** (via IDs) across multiple steps, making workflows like follow-up queries or document retrieval precise and reliable.

C. A JSON array of document titles extracted from the search results. Incorrect.

Titles alone are ambiguous and not stable identifiers, making it difficult for agents to reliably act on specific documents.

D. More detailed human-readable descriptions including the size and authors. Incorrect.

Helpful for users, but still **unstructured** and not suitable for precise multi-step agent operations.

Question: 20 **Exam Author**

Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a search_connectors(description) tool that finds matching connectors, but in testing agents frequently skip searching and call connectors directly (often incorrectly), or search select wrong connectors from the filtered results.How should you design the tool composition pattern to address both issues?

A. Design connectors with built-in compatibility validation that return descriptive errors for mismatched requests.

B. Design a find_and_execute(description, params) composite tool that searches and immediately executes the best matching connector.

C. Enhance all connector descriptions with detailed usage samples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.

D. Design search_connectors to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

Answer: D

Explanation:

A. Design connectors with built-in compatibility validation that return descriptive errors for mismatched requests. Incorrect.

This helps with error handling after a wrong choice is made, but does **not improve initial tool selection accuracy**.

B. Design a find_and_execute(description, params) composite tool that searches and immediately executes the best matching connector. Incorrect.

This removes transparency and control, making debugging harder and preventing the agent from reasoning about tool choice.

C. Enhance all connector descriptions with detailed usage samples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow. Incorrect.

While helpful, this still relies on the agent to follow instructions and does not enforce correct behavior, especially at scale with 50+ tools.

D. Design search_connectors to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered. Correct.

This enforces the **search-first pattern** by limiting available tools initially and **reducing the decision space**, improving both discovery and correct selection.

Question: 21 **Exam Author**

Your publish article tool calls an external CMS API that occasionally returns transient errors (network timeouts, 503s) and non-transient errors (403 permission denied, 422 validation failure).Currently, every error is returned directly to the agent, which leads to the agent retrying non-transient errors and wasting turns on failures that will never succeed. How should you partition error-handling responsibility between the tool implementation and the agent?

A. Handle all errors inside the tool: Implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.

B. Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation fallures) to the agent with descriptive messages so it can

take corrective action.

C. Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times-keeping the tool implementation stateless and simple.

D. Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable-try again later" message, shielding the agent from error complexity.

Answer: B

Explanation:

A. Handle all errors inside the tool: Implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.
Incorrect.

This wastes time retrying **non-transient errors** (e.g., 403, 422) that will never succeed and hides useful feedback from the agent.

B. Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation failures) to the agent with descriptive messages so it can take corrective action. Correct.

This cleanly separates responsibility:

Tool handles **recoverable/transient issues** automatically
Agent receives **actionable errors** it can fix (permissions, input validation)

C. Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times — keeping the tool implementation stateless and simple. Incorrect.

This pushes retry logic to the agent, leading to **inefficient behavior and wasted turns**.

D. Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable — try again later" message, shielding the agent from error complexity. Incorrect.

This removes critical detail, preventing the agent from taking **corrective actions** when possible.

Question: 22 **Exam Author**

Your remove_team_member tool uses a dry_run: boolean parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step in 15% of calls by calling with dry_run=false directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?

A. Add server-side validation that permits dry_run=false only when a dry_run=true call with identical parameters occurred within the past 60 seconds.

B. Replace with two tools: preview_remove_member returns impact details and a single-use confirmation token; execute_remove_member requires that token, binding execution to the specific previewed action.

C. Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.

D. Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with dry_run=true first and wait for user confirmation before calling with dry_run=false.

Answer: B

Explanation:

A. Add server-side validation that permits dry_run=false only when a dry_run=true call with identical

parameters occurred within the past 60 seconds. Incorrect.

This approach is brittle because it depends on timing and does not guarantee that the user actually reviewed or confirmed the preview.

B. Replace with two tools: preview_remove_member returns impact details and a single-use confirmation token; execute_remove_member requires that token, binding execution to the specific previewed action. Correct.

This enforces the correct workflow at the system level by requiring a valid preview step and tying execution to an explicit confirmation, making bypass impossible.

C. Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools. Incorrect.

This depends on orchestration behavior and is not strictly enforced, so it can still be bypassed or misconfigured.

D. Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with dry_run=true first and wait for user confirmation before calling with dry_run=false. Incorrect.

Instruction-based approaches are not reliable for enforcement, as demonstrated by the existing bypass rate.

Question: 23 **Exam Author**

Your expense reimbursement agent processes employee requests using a process reimbursement tool. Company policy requires that reimbursements above $500 must be approved before funds are disbursed. The agent handles hundreds of requests daily, and you need the threshold enforcement to be tamper-proof regardless of how the agent is prompted ensures the $500 approval threshold cannot be bypassed?

A. The process reimbursement tool accepts an approved by manager parameter. The system prompt instructs the agent to only set this to true after confirming that a manager approved the request. A nightly audit script reviews all reimbursements where approved by manager was set to true.

B. Provide two tools: auto reimburse (hard-coded limit of $500) and manager approval. Include detailed system prompt instructions telling the agent to check the amount and use the appropriate tool. Add a Post ToolUse hook that logs which tool was called for auditing.

C. The process reimbursement tool accepts amount and details, and internally enforces the threshold; amounts <$500 are auto-disbursed and the tool returns a success confirmation. Amounts >$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.

D. Implement the threshold check in a PreToolUse hook that inspects the amount parameter before process reimbursement executes. If the amount exceeds $500, the hook modifies the context to add a requires approval: true flag, which the tool checks before disbursing.

Answer: C

Explanation:

A. The process reimbursement tool accepts an approved by manager parameter. The system prompt instructs the agent to only set this to true after confirming that a manager approved the request. A nightly audit script reviews all reimbursements where approved by manager was set to true. Incorrect.

This relies on the agent following instructions and post-hoc auditing, which is not tamper-proof and allows bypass at execution time.

B. Provide two tools: auto reimburse (hard-coded limit of $500) and manager approval. Include detailed system prompt instructions telling the agent to check the amount and use the appropriate tool. Add a Post ToolUse hook that logs which tool was called for auditing. Incorrect.

Again depends on agent behavior and correct tool selection. Logging helps auditing but does not prevent misuse.

C. The process reimbursement tool accepts amount and details, and internally enforces the threshold; amounts <$500 are auto-disbursed and the tool returns a success confirmation. Amounts >$500 cause the tool to create a pending approval request and return a status indicating manager review is pending. Correct.

This enforces the rule **inside the tool itself**, making it impossible to bypass regardless of how the agent is prompted.

D. Implement the threshold check in a PreToolUse hook that inspects the amount parameter before process reimbursement executes. If the amount exceeds $500, the hook modifies the context to add a requires approval: true flag, which the tool checks before disbursing. Incorrect.

PreToolUse hooks can be bypassed or misconfigured and still rely on downstream logic. Enforcement should reside directly within the tool for full reliability.

Question: 24 **Exam Author**

Your order management system requires tools for three distinct operations: issuing refunds (requires amount and reason), canceling orders (requires reason), and res (requires shipping address). Each operation shares an order id parameter but has different additional requirements. You notice during testing that with your current frequently omits required parameters or includes irrelevant ones. What design change will most effectively improve parameter accuracy?

A. Split into three separate tools (each defining only the parameters required for that specific operation.

B. Keep one unified tool with all parameters marked optional, but add few-shot examples in the system prompt showing correct parameter combinations for each operation.

C. Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund".

D. Keep one unified tool with a nested operation object parameter whose internal structure varies by operation type, documented in the tool description.

Answer: A

Explanation:

A. Split into three separate tools (e.g., issue_refund, cancel_order, reship_order), each defining only the parameters required for that specific operation. Correct.

This reduces ambiguity and ensures the agent only sees **relevant parameters per operation**, leading to much higher accuracy.

B. Keep one unified tool with all parameters marked optional, but add few-shot examples in the system prompt showing correct parameter combinations for each operation. Incorrect.

Examples help, but the schema remains ambiguous, so errors will still occur.

C. Keep one unified tool but add JSON Schema if-then-else conditionals to enforce that parameters like amount are required only when the operation type is "refund". Incorrect.

While technically valid, this increases complexity and is less reliable than simply separating tools.

D. Keep one unified tool with a nested operation object parameter whose internal structure varies by operation type, documented in the tool description. Incorrect.

This adds complexity and cognitive load, making it harder for the agent to consistently provide correct parameters.

Question: 25 **Exam Author**

Your portfolio value tool returns the total value of a user's investment portfolio. You're deciding between returning a structured JSON object with explicit fields versus returning information as a formatted text string. What is the primary advantage of using structured output with defined fields?

A. Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs.

B. The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations.

C. Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values.

D. JSON schemas automatically validate that the underlying API returned correct data before the agent processes it.

Answer: B

Explanation:

A. Structured JSON consumes significantly fewer tokens than natural language, substantially reducing API costs. Incorrect.

Token usage depends on the content; JSON is not inherently more compact than text and may sometimes use more tokens.

B. The agent can reliably extract specific values without parsing free form text, reducing errors in subsequent operations. Correct.

Structured output provides **clear, predictable fields**, making it easy for the agent to use the data accurately in downstream steps.

C. Structured JSON is processed deterministically by the model, significantly improving accuracy when extracting values. Incorrect.

The model is still probabilistic; JSON improves structure, but not deterministic processing.

D. JSON schemas automatically validate that the underlying API returned correct data before the agent processes it. Incorrect.

Schemas define structure, but they do not guarantee correctness of the actual data returned by the API.

Question: 26 **Exam Author**

Your scheduling agent uses get_available_slots(date, provider_id) to retrieve open appointment times, then book_appointment(provider_id, slot_time, patient_id) to reserve a slot. tickets show that 15% of booking attempts fall with "slot no longer available" because another user booked the slot between the availability check and the booking call. How should you r these tools?

A. Modify book_appointment to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a di time.

B. Keep both tools but add retry logic to the agent's system prompt, instructing it to call get_available_slots again and select a different time if booking fails.

C. Add a hold_slot(provider_id, slot_time) tool that creates a 60 second temporary reservation, requiring the agent to call it between checking availability and booking.

D. Combine both tools into a single find_and_book_appointment that atomically checks availability and books, returning either the confirmed booking or available alternatives.

Answer: D

Explanation:

A. Modify book_appointment to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a different time. Incorrect.

This improves recovery but does not fix the **race condition** between availability check and booking.

B. Keep both tools but add retry logic to the agent's system prompt, instructing it to call get_available_slots again and select a different time if booking fails. Incorrect.

This still suffers from the same race condition and relies on agent behavior rather than fixing the underlying issue.

C. Add a hold_slot(provider_id, slot_time) tool that creates a 60 second temporary reservation, requiring the agent to call it between checking availability and booking. Incorrect.

This reduces the issue but introduces additional complexity and still requires multiple steps that can fail.

D. Combine both tools into a single find_and_book_appointment that atomically checks availability and books, returning either the confirmed booking or available alternatives. Correct.

This eliminates the race condition by making the operation **atomic**, ensuring consistency and reliability.

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

But Wait

I wanted to let you know that there is more content available in the full version.
The full paper contains additional sections and information that you may find helpful, and I encourage you to download it to get a more comprehensive and detailed view of all the subject matter.

Download Full Version Now

Future is Secured

100% Pass Guarantee

24/7 Customer Support

Mail us - support@examauthor.com

Verified Updates

Lifetime Updates!

Total: **85 Questions**

Link: https://examauthor.com/papers/anthropic/claude-certified-architect

1) When allocating tools to agents in a multi-agent system, what is the architectural impact of giving a single agent access to an excessive number of tools (e.g., 18 tools)?

A. The agent automatically splits itself into multiple subagents to handle the load.

B. Tool selection reliability degrades significantly due to increased decision complexity and ambiguity among similar tools.

C. The Claude API enforces a hard limit and rejects requests with more than 10 tools.

D. The model's temperature automatically drops to 0.0 to compensate for the large tool space.

Answer: B

2) The agent attempts to call a lookup_order tool, but the backend database is down due to a timeout. What structured error response should the tool return to the agent?

A. An empty JSON array [J so the agent can smoothly transition to an alternative tool.

B. A structured object containing isError: true, an error category of 'timeout', isRetryab1e: true, and context of what was attempted.

C. A generic string " Operation failed"} to avoid leaking infrastructure details to the LLM.

D. {"error : " A fatal exception that immediately terminates the agentic loop.

Answer: B

3) When using tool_use with a strict JSON schema for data extraction, what does the Claude API explicitly guarantee?

A. Semantic correctness of the extracted data.

B. Structural compliance, effectively eliminating JSON syntax errors.

C. 100% factual accuracy of the extracted entities.

D. Elimination of the need for downstream data validation.

Answer: B

www.shapingpixel.com

4) What is the recommended strategy for an agent to trace function usage across wrapper modules in an unfamiliar codebase?

A. Read all project files upfront to build a comprehensive context map in memory.

B. Use the Edit tool to temporarily inject logging statements into every module and observe the output.

C. Start with Grep to identify all exported names and entry points, search for each name across the codebase, and then use Read to trace specific flows.

D. Use Glob to find all . js and .ts files and concatenate them using Bash to preserve token sequence.

Answer: C

5) You have testing conventions that must be uniformly applied, but your test files are distributed alongside source code files (e.g., Button. test. tsx next to Button. tsx). Why is creating a rule file in . claude/rules/ better than modifying the root CLAUDE. md?

A. The root CLAUDE .md cannot process testing frameworks like Jest or Vitest.

B. Consolidating all conventions in the root CLAUDE .md requires Claude to infer which sections apply to which files, making the application of rules probabilistic rather than deterministic.

C. Rules in . claude/rules/ can natively execute shell commands prior to loading, unlike the root configuration.

D. The root CLAUDE .md is strictly reserved for environment variable expansion.

Answer: B

6) A subagent attempts to query an external database but encounters a timeout. How should this error be propagated back to the coordinator agent?

A. The subagent should silently return an empty JSON array to allow the workflow to proceed smoothly.

B. Return a generic 'Operation failed' string to prevent leaking infrastructure details.

C. Return structured error context, including the failure type (access failure), what was attempted, and partial results.

D. Throw a fatal exception to immediately terminate the entire research workflow.

Answer: C

www.shapingpixel.com

7) Which of the following tasks is the MOST appropriate use case for Direct Execution rather than Plan Mode?

A. Changing the state management library from Redux to Zustand across a large React application.

B. Designing an implementation approach for a new API integration with unknown dependencies.

C. Fixing a specific bug with a clear stack trace in a single TypeScript file.

D. Restructuring the database schema and updating all corresponding repository models.

Answer: C

8) During a tool execution step, your agent's lookup_customer tool fails to connect to the external database due to a timeout. What is the architecturally correct way to handle this error and propagate it back to Claude?

A. Return a structured error response with isError: true, specify the error category (e.g., timeout), indicate if it is retryable, and provide context about what was attempted.

B. Silently return an empty JSON array [J so the agent can smoothly transition to trying an alternative lookup method.

C. Return a generic string like { "error" : " Operation failed" } to prevent leaking internal database infrastructure details to the LLM.

D. Throw a fatal system exception to immediately terminate the agentic loop and trigger a fallback to a human operator.

Answer: A

9) To optimize a structured data extraction pipeline, you want to route only the most uncertain documents to your limited pool of human reviewers. Which method provides the most reliable calibration for this routing?

A. Relying on the model's self-reported confidence score in the conversational text output.

B. Routing any document that triggers a validation-retry loop more than once.

C. Having the model output field-level confidence scores, and calibrating those thresholds using a labeled validation set.

D. Escalating documents based solely on the aggregate accuracy metric of the entire pipeline.

Answer: C

www.shapingpixel.com

10) A application features a tool designed to fetch a user's profile. During execution, the tool fails to connect to the database due to a timeout. Why is silently returning an empty array [ ] in this scenario considered a catastrophic anti-pattern?

A. It prevents the use of few-shot prompting in the next iteration.

B. It causes the agent to misinterpret an access failure (could not check) as a valid empty result, wrongly concluding the user does not exist.

C. It forces the model to ignore all subsequent tool calls in the session.

D. It causes the Claude API to throw a 400 Bad Request error due to missing required schema fields.

Answer: B

11) In Model Context Protocol architecture, what is the primary distinction between an MCP Tool and an MCP Resource?

A. Resources execute functions that perform backend mutations, whereas Tools expose data catalogs.

B. Resources expose static or templated data that clients can fetch to inject into prompts, whereas Tools are executable functions that perform actions.

C. Resources are hosted locally via stdio, whereas Tools are hosted remotely via HTTP.

D. Resources provide pre-built conversation templates, whereas Tools provide context history.

Answer: B

12) Which of the following should be included in structured error context propagated to the coordinator to enable intelligent recovery decisions? (Choose 2 correct answers)

A. The failure type and what was attempted.

B. The full uncompressed conversation history of the subagent.

C. Any partial results obtained before the failure.

D. The model's internal token counts and neural weights.

Answer: A,C

www.shapingpixel.com

13) A developer at builds an MCP server to standardize repetitive actions for the engineering team. They want to provide ready-to-use message sequences with specific formatting instructions that bypass the need for users to write them from scratch. Which MCP component fulfills this requirement?

A. MCP Subagents

B. MCP Prompts

C. MCP Tools

D. MCP Resources

Answer: B

14) You are building a customer support resolution agent using the Claude Agent SDK. The agent is currently escalating simple requests like password resets if the customer uses angry language. How should you fix this escalation logic?

A. Lower the sentiment analysis threshold so that the agent only escalates if explicit profanity is detected.

B. Implement escalation based on objective criteria such as policy gaps, capability limits, or explicit human requests, rather than customer sentiment.

C. Issue an automatic partial refund to calm the customer down before processing the password reset.

D. Use a PreToolUse hook to normalize the customer's text input to a neutral tone before Claude processes it.

Answer: B

15) In an extended codebase exploration session, Claude begins returning inconsistent answers and forgetting specific classes discovered earlier. Which combination of features best addresses this context degradation?

A. Utilizing / clear and increasing the temperature parameter to 1.0.

B. Relying entirely on the Message Batches API for asynchronous context processing.

C. Using scratchpad files to persist key findings and using the /compact command to reduce context usage.

D. Spawning parallel subagents without providing them explicit context.

Answer: C

www.shapingpixel.com

16) A developer wants to create a reusable behavior for performing extensive, multi-step codebase refactoring. The behavior needs to explore the codebase without polluting the main conversation's context window. Which extension mechanism should be used?

A. A custom command in . claude/commands/.

B. A project-scoped CLAUDE .md configuration file.

C. A skill defined in a SKILL.md file with context: fork in its frontmatter.

D. A PostToolUse hook running a background Bash script.

Answer: C

17) A pull request modifies 14 files across a repository. A single-pass automated review produces inconsistent results, catching deep bugs in some files while missing obvious ones in others. How should the review architecture be redesigned to fix this?

A. Switch to a higher-tier model with a larger context window to process all files simultaneously.

B. Require developers to manually split the PR into smaller submissions of 3-4 files.

C. Split the review into focused passes: analyze each file individually for local issues, followed by a separate integration pass for cross-file data flow.

D. Run three independent reviews on the full PR and only flag issues that reach consensus.

Answer: C

18) Which of the following scenarios is the most appropriate use case for selecting Direct Execution rather than Plan Mode?

A. Restructuring a monolithic application into discrete microservices.

B. Evaluating multiple integration approaches for a new third-party payment provider.

C. Adding a date validation conditional to a single, specific utility function.

D. Redesigning the database schema and updating all corresponding data access objects.

Answer: C

www.shapingpixel.com

19) A developer needs to migrate a monolithic application into microservices, requiring changes across 45+ files, updates to module dependencies, and new architectural boundaries. Which execution approach in Claude Code is recommended for this task?

A. Use Direct Execution to incrementally modify files one by one until the migration is complete.

B. Use the /compact command heavily to fit all 45 files into the context window for a single-pass Direct Execution.

C. Use Plan mode to explore the codebase, map dependencies, and design an implementation strategy before committing to file modifications.

D. Write a static Prompt Chain that forces the agent to read, refactor, and save each file sequentially.

Answer: C

20) Using the @anthropic-ai/claude-agent-sdk, you want to programmatically narrow the set of tools available to a specific query loop to prevent the agent from making unintended modifications. Which configuration is used to restrict tools in the SDK?

A. Pass the allowedTools: [" Read , '" ' Glob"] array inside the options object of the query() function.

B. Inject a permissions: read-only tag into the top-level AgentDefinition.

C. Set tool choice: "none" to force the model to only use search tools.

D. Edit the . mcp. j son file to globally disable the write tools for the entire project.

Answer: A

21) You are building a classifier to categorize customer reviews, but the model struggles with ambiguous scenarios like sarcasm and mixed sentiments. You decide to use few-shot prompting. What is the recommended best practice for this scenario?

A. Provide exactly 1 example of a standard positive review to establish the JSON format.

B. Provide 8 to 10 examples to ensure every possible sentiment category is covered comprehensively.

C. Provide 2 to 4 examples, ensuring that at least one example specifically addresses an ambiguous edge case like sarcasm.

D. Avoid few-shot prompting and instead use a detailed system prompt explaining the exact definition of sarcasm.

Answer: C

www.shapingpixel.com

22) When designing the role of the coordinator agent in a multi-agent system, which of the following practices are recommended? Choose 2 correct answers.

A. Routing all inter-subagent communication through the coordinator to maintain observability and consistent error handling.

B. Analyzing query requirements and dynamically selecting which subagents to invoke rather than always routing through the full pipeline.

C. Forcing all queries through a flat architecture pipeline to maximize cross-agent state sharing.

D. Decomposing broad research topics into overly narrow subtasks to save context tokens at the risk of coverage gaps.

Answer: A,B

23) Which of the following statements correctly describe best practices for integrating Claude Code into a CI/CD pipeline? Choose 2 correct answers.

A. Using the Message Batches API for blocking pre-merge checks to maximize cost savings.

B. Using the -p flag for non-interactive execution so the pipeline does not hang waiting for user input.

C. Separating the code generation session from the code review session to ensure the reviewer evaluates the code objectively.

D. Running Claude Code in interactive mode to allow developers to answer clarifying questions during the build process.

Answer: B,C

24) Which of the following scenarios represent appropriate, architecturally sound use cases for Agent SDK hooks? Choose 2 correct answers.

A. Normalizing heterogeneous numeric status codes from different MCP tools before appending them to the conversation context.

B. Guaranteeing that a model-extracted JSON object contains no semantic reasoning errors.

C. Enforcing the conversational tone, politeness, and persona the agent should adopt.

D. Intercepting outgoing tool calls to deterministically block policy-violating actions, such as unauthorized financial transactions.

Answer: A,D

25) While working in Claude Code, you press Shift + Tab to enter Plan Mode. Which subagent is typically associated with isolating the verbose discovery phase to prevent context window exhaustion during planning?

A. The Review subagent

B. The Write subagent

C. The Explore subagent

D. The Plan-Executor subagent

Answer: C

26) When designing a multi-agent research system for a enterprise application, which architectural pattern is recommended to manage parallel research tasks effectively?

A. A flat architecture where all agents share a global state and full conversation history to maximize context visibility.

B. A hub-and-spoke architecture where a central coordinator delegates tasks to specialized subagents with isolated contexts.

C. A sequential pipeline architecture where each agent passes its entire context and output directly to the next agent in the chain.

D. A monolithic architecture utilizing a single agent loaded with all available research and synthesis tools.

Answer: B

27) A developer implements an escalation rule that triggers human review whenever the customer uses angry language. Why is this considered an anti-pattern?

A. Sentiment does not correlate with task complexity; even angry customers may have simple requests that can be handled autonomously.

B. The sentiment threshold should only apply to explicit profanity to avoid false positives.

C. The agent should always issue an automatic partial refund to calm the customer before escalating.

D. The agent lacks a PostToolUse hook to properly process text sentiment scores.

Answer: A

www.shapingpixel.com

28) When implementing multi-pass review architectures for automated CI/CD pipelines, why is configuring the exact same Claude SDK session to review its newly generated code considered a major anti-pattern?

A. It causes the token limit of the context window to exceed its maximum capacity.

B. The Message Batches API does not support synchronous self-review workflows.

C. Tools invoked in the first pass are permanently locked and cannot be reused in the second pass.

D. The reviewer retains the reasoning context from the generation phase, creating confirmation bias and reducing the likelihood of catching subtle flaws.

Answer: D

29) Which of the following scenarios are best suited for prompt chaining (fixed sequential pipelines) rather than dynamic adaptive decomposition? Choose 2 correct answers.

A. A predictable, multi-aspect security review where the sequence of code analysis steps is static and predefined.

B. An open-ended investigation into an undocumented legacy API where responses guide the next queries.

C. A data processing pipeline where raw text is summarized, then translated, and finally formatted into JSON.

D. A monolith-to-microservices architectural migration involving dozens of interacting dependencies.

Answer: A,C

30) What is the primary architectural advantage of using glob-pattern rules in a . claude/rules/ file rather than placing a CLAUDE. md file in a subdirectory?

A. Glob patterns bypass the Model Context Protocol (MCP) server requirements for fetching dynamic rules.

B. Glob patterns allow conventions to apply to files by type regardless of their directory location, which is essential for conventions that span multiple directories.

C. Glob patterns automatically apply the rules to the global system-level /etc/c1aude/CLAUDE .md configuration.

D. Glob patterns disable Claude's semantic inference, forcing it to use exact keyword matching for all prompts.

Answer: B

www.shapingpixel.com

31) After updating a tool description to perfectly define its purpose and boundaries, Claude still occasionally misroutes specific requests to an older, less appropriate tool. What hidden factor is most likely causing this unintended tool association?

A. The tool_choice parameter is set to auto instead of any.

B. Keyword-sensitive instructions embedded in the system prompt are overriding the tool descriptions.

C. The Model Context Protocol automatically defaults to tools with shorter alphabetical names.

D. The agent is executing a PostToolUse hook that rewrites the tool execution payload.

Answer: B

32) Which statements correctly describe the relationship between JSON schemas and validation loops in structured extraction? Choose 2 correct answers.

A. tool_use eliminates schema syntax errors, greatly reducing the need for syntax-focused retries.

B. Semantic errors (e.g., values placed in wrong fields) are prevented entirely by strict JSON schemas.

C. Validation loops are necessary to catch semantic errors, such as line items failing to sum to the extracted total.

D. Validation retries are most effective when the required information is completely missing from the document.

Answer: A,C

33) A developer asks the agent to 'read the project configuration file to determine the database connection string'. Which built-in tool represents the correct architectural choice for this task?

A. "database", "config. j son

B. Bash("cat config. j son")

C. Grep(Glob (" config. j son ")

D. Read (" config. j son

Answer: D

www.shapingpixel.com

34) A pull request modifies 14 files across a stock tracking module. When analyzing all files in a single pass, Claude provides superficial comments, misses obvious bugs, and produces contradictory feedback. How should you redesign the multi-pass review architecture to resolve this?

A. Switch to a higher-tier model with a larger context window so all 14 files receive adequate attention in one pass.

B. Require developers to manually split large PRs into smaller submissions of 3-4 files before triggering the automated review.

C. Run three independent single-pass reviews on the full PR and only flag issues that achieve consensus across at least two runs.

D. Split the review into focused passes: analyze each file individually for local issues, followed by a separate integration-focused pass for cross-file data flow.

Answer: D

35) A developer needs to safely explore two completely different architectural approaches for refactoring a database module without polluting the main conversation's context. Which Claude Code session command should be used?

A. / compact

B. fork_session

C. --resume

D. / clear

Answer: B

36) Which of the following statements represent correct principles of tool distribution and interface design? Choose 2 correct answers.

A. Increasing the total number of tools given to a single coordinator agent inherently improves its decision-making accuracy.

B. Grouping related tools and assigning them to specialized subagents improves tool selection reliability.

C. Setting tool_choice: "auto" forces Claude to always execute at least one tool.

D. Providing detailed input formats, edge cases, and boundaries in tool descriptions prevents the model from misrouting requests between similar tools.

Answer: B,D

www.shapingpixel.com

37) To measure ongoing error rates in high-confidence extractions and detect novel error patterns, which methodology should be implemented?

A. Stratified random sampling.

B. Exhaustive 100% human review.

C. Aggregate accuracy averaging.

D. Prompt-based sequential pipelining.

Answer: A

38) Which of the following approaches represent valid techniques for designing prompts that improve precision and reduce false positives?
Choose 2 correct answers.

A. Defining explicit, measurable criteria instead of vague instructions.

B. Using vague instructions like "make it better" to give the model maximum creative freedom.

C. Providing concrete code examples distinguishing acceptable patterns from genuine issues.

D. Instructing the model to self-evaluate and drop findings if its conversational confidence is low.

Answer: A,C

39) Before automating the extraction of high-confidence fields in a quicktechie.com processing system, what validation step is crucial?

A. Validating the JSON schema syntax using a PreToolUse hook.

B. Validating accuracy segmented by document type and specific field to verify consistent performance across all segments.

C. Confirming consensus through 3 separate monolithic review passes.

D. Checking that the model's self-reported sentiment analysis is positive.

Answer: B

www.shapingpixel.com

39) Before automating the extraction of high-confidence fields in a quicktechie.com processing system, what validation step is crucial?

A. Validating the JSON schema syntax using a PreToolUse hook.

B. Validating accuracy segmented by document type and specific field to verify consistent performance across all segments.

C. Confirming consensus through 3 separate monolithic review passes.

D. Checking that the model's self-reported sentiment analysis is positive.

Answer: B

41) Production logs indicate that Claude is frequently misrouting requests to an analyze_content tool when the analyze_document tool should have been selected. Both tools currently have simple, one-sentence descriptions. What is the most effective architectural fix?

A. Provide negative few-shot examples in the system prompt instructing Claude not to confuse the two tools.

B. Rename the tools to eliminate functional overlap and expand descriptions to explicitly state boundaries explaining when to use each.

C. Use a PreToolUse hook to programmatically redirect analyze_content calls to analyze_document.

D. Increase the model's temperature parameter to encourage more diverse tool selection.

Answer: B

42) Which of the following represent best practices for managing context during a large codebase exploration? (Choose 2 correct answers)

A. Relying solely on the model's extended thinking capabilities instead of external scratchpad files.

B. Summarizing key findings from one exploration phase and injecting those summaries into the initial context for the next phase's subagents.

C. Passing the full, uncompressed conversation history of the coordinator to every subagent.

D. Using subagent delegation to isolate verbose search results and file reads from the coordinator's context.

Answer: B,D

www.shapingpixel.com

43) When implementing path-specific rules for conditional convention loading in Claude Code, which of the following statements accurately describe the architecture? Choose 2 correct answers.

A. Path-specific rules are defined using YAML frontmatter with a paths array containing glob patterns.

B. Path-specific rule files must be stored exclusively in the user-level ~/. claude/rules/ directory to prevent team-wide conflicts.

C. They are ideal for applying coding conventions to file types (like **/*. test. tsx) that are distributed across many different subdirectories.

D. Path-specific rules automatically execute a PreToolUse hook to format files before they are loaded into context.

Answer: A,C

44) In a multi-pass review architecture, how can you effectively automate the routing of identified findings for human review calibration?

A. Run verification passes where the model outputs field-level confidence scores alongside findings to enable calibrated review routing.

B. Rely strictly on the model's self-reported conversational confidence score in the output.

C. Escalate any finding that flags more than 5 lines of code.

D. Use the Message Batches API to delay findings until the end-of-week manual review.

Answer: A

45) When extracting financial summaries from unstructured documents, how can you design a self-correction validation flow directly within the extraction process?

A. Extract a calculated_total alongside the stated_total to programmatically flag discrepancies.

B. Increase the max_tokens limit to give the model more space to reason about the math.

C. Provide few-shot examples of flawless mathematical calculations.

D. Execute the extraction inside a PreToolUse hook.

Answer: A

www.shapingpixel.com

46) When implementing structured data extraction for a application, what does utilizing tool_use with a strict JSON schema explicitly guarantee?

A. Both structural compliance and semantic correctness of the extracted data.

B. Structural compliance, ensuring valid JSON syntax and required fields, but not semantic accuracy.

C. Elimination of all hallucinations and inaccurate data extraction.

D. Automatic retry loops if the extracted values fail business logic validation.

Answer: B

47) Which of the following represent architecturally sound strategies for managing context effectively across long documents or multi-agent handoffs? Choose 2 correct answers.

A. Sharing the full coordinator conversation history with every subagent to guarantee maximum context.

B. Placing key findings summaries at the beginning of aggregated inputs to mitigate position effects.

C. Requiring subagents to output structured claim-source mappings to support accurate downstream synthesis.

D. Replacing specific numeric constraints in long contexts with generalized natural language summaries.

Answer: B,C

48) Your MCP server exposes a Templated Resource with the URI pattern docs://documents/{doc_id}. How does the MCP Python SDK handle this pattern?

A. It requires the client to pass the doc_id via an HTTP POST body.

B. It automatically parses the doc_id from the URI and passes it as a keyword argument to the underlying resource function.

C. It treats the entire URI as a static string and expects the function to manually parse it.

D. It requires the model to write a regex to extract the doc_id before passing it to the server.

Answer: B

www.shapingpixel.com

49) Which of the following statements accurately describe the characteristics and constraints of the Message Batches API? Choose 2 correct answers.

A. It provides a guaranteed I-hour latency SLA for enterprise tier accounts.

B. It offers a 50% cost savings compared to standard synchronous API calls.

C. It uses custom_id fields to correlate asynchronous requests with their respective responses.

D. It is optimized to support interactive, real-time chatbot workflows at scale.

Answer: B,C

50) An MCP tool processes financial refunds. A policy violation occurs because the requested amount exceeds the user's maximum allowed limit. What is the most architecturally sound way the tool should respond?

A. Return a generic "Operation failed" string to avoid leaking internal policy limits.

B. Throw a fatal system exception to halt execution.

C. Return structured error metadata with isError: true, a human-readable context description of the policy violation, and isRetryable: false.

D. Return an empty response payload so the agent can seamlessly try a different tool.

Answer: C

51) How should field-level confidence scores from a model be utilized to optimize a human-in-the-loop workflow?

A. By trusting the model's self-reported conversational confidence score to bypass human review.

B. By calibrating the scores using labeled validation sets to effectively route review attention to low-confidence extractions.

C. By using them to automatically increase the max_tokens limit on subsequent runs.

D. By setting tool_choice: 'any" whenever confidence drops below 0.5.

Answer: B

www.shapingpixel.com

52) When building the raw tool-handling conversation loop with the API/SDK, Claude may return a ToolIJse block. To properly continue the multi-turn agentic loop, what are the requirements for formatting and returning the result of the executed function to Claude? Choose 2 correct answers.

A. The result must be appended to the messages array inside a user role message containing a tool_result block.

B. The tool result block must include a that matches the ID generated by Claude's initial tool request.

C. The result must be passed back inside an assistant role message with an is_error'. false flag.

D. The tool result must strictly be a serialized JSON object; raw text strings will automatically trigger a loop termination.

Answer: A,B

53) You are building a code review tool and instruct the model to "be thorough and flag everything suspicious." Why is this considered a prompt engineering anti-pattern?

A. It forces the model to bypass tool usage.

B. It leads to over-flagging and alert fatigue, which erodes developer trust in the system.

C. It causes the context window to rapidly exceed its maximum token limit.

D. It guarantees semantic correctness but compromises schema compliance.

Answer: B

54) When tasked with fixing a bug on line 42 of an existing large TypeScript file, why must the agent choose the Edit tool instead of the Write tool?

A. The Edit tool executes significantly faster because it bypasses the Model Context Protocol (MCP) routing layer.

B. The Write tool replaces the entire file content, which risks the catastrophic loss of unchanged content if the agent does not rewrite the file perfectly.

C. The Write tool requires an active MCP server connection, whereas the Edit tool is executed locally.

D. The Edit tool automatically triggers a pre-configured validation hook to run ESLint.

Answer: B

www.shapingpixel.com

55) When designing subagents to handle conflicting data from different sources during the final synthesis phase, which of the following are recommended practices? Choose 2 correct answers.

A. Silently average the conflicting numerical values to provide a balanced estimate.

B. Require subagents to output structured claim-source mappings.

C. Preserve information provenance including the source, confidence level, and timestamp.

D. Arbitrarily select the value from the most recently updated database to resolve the conflict.

Answer: B,C

56) When natural language descriptions of a complex data transformation continually produce inconsistent results from Claude, what is the most effective iterative refinement technique to correct this behavior?

A. Appending "BE EXTREMELY PRECISE" to the end of the system prompt.

B. Providing 2-3 concrete input/output examples to clarify the exact expected transformation requirements.

C. Increasing the max_tokens parameter to allow for a longer reasoning chain.

D. Changing the tool_choice setting to auto.

Answer: B

57) In an automated CI/CD code review workflow at quicktechie.com, why is it considered a major architectural anti-pattern to have the same Claude session that generated the code also perform the self-evaluation review?

A. It exceeds the maximum token limit for the context window during the generation phase.

B. The reviewer retains the reasoning context and assumptions from the generation phase, creating confirmation bias.

C. The Message Batches API does not support synchronous self-review workflows.

D. Tools invoked during the first pass are permanently locked and cannot be reused in the review pass.

Answer: B

www.shapingpixel.com

58) What is the only reliable, deterministic signal in the Claude Agent SDK to evaluate whether an agentic loop should continue executing tools or terminate?

A. Parsing the assistant's natural language text output for completion keywords like 'task complete'.

B. Inspecting the stop_reason field of the response for tool_use versus end_turn.

C. Monitoring the total token count and exiting the loop when it reaches the context window limit.

D. Setting a hard maximum iteration cap of 5 loops to prevent infinite execution.

Answer: B

59) When implementing a multi-agent research system for a enterprise application, a coordinator agent must delegate a market analysis task to a specialized subagent. What is the correct approach for passing context to this subagent?

A. Share the entire coordinator conversation history to ensure the subagent has maximum context.

B. Allow the subagent to access a shared global state object where the coordinator writes its findings.

C. Pass only the specific, explicit context relevant to the subagent's assigned subtask in the prompt.

D. Use the /compact command to summarize the coordinator history before passing it to the subagent.

Answer: C

60) Which of the following strategies are highly recommended for integrating Claude into a robust test generation workflow? Choose 2 correct answers.

A. Document testing standards, valuable test criteria, and available fixtures in the project-level CLAUDE .md.

B. Rely on aggregate test coverage metrics to determine if the generated tests are semantically correct.

C. Share specific test failure outputs with Claude to guide progressive improvement of the implementation.

D. Execute tests automatically within a PreToolUse hook to prevent Claude from saving failing code.

Answer: A,C

www.shapingpixel.com

61) In a multi-agent research system, subagent A retrieves revenue data from a verified financial database, while subagent B extracts conflicting revenue data from an outdated PDF. How should the system handle this uncertainty before presenting it to a human reviewer?

A. Silently drop the PDF data and return the database value since databases are always correct.

B. Average the two values to provide a balanced estimate to the human reviewer.

C. Maintain structured claim-source mappings with information provenance (source, confidence, timestamp) so the coordinator or human can make an informed resolution.

D. Restart both subagents using a higher temperature setting to generate a consensus value.

Answer: C

62) A developer grants a single Claude agent access to 18 different tools for a complex refactoring task. Production logs indicate that the agent frequently selects the wrong tool. What is the most effective architectural fix for this issue?

A. Expand the tool descriptions to include more exhaustive examples and edge cases for all 18 tools.

B. Reduce the number of tools per agent to 4-5 by distributing the rest across specialized subagents.

C. Implement a routing layer that pre-selects tools based on keyword matching in the user's prompt.

D. Switch to a higher-tier model with a larger context window to better process the 18 tool schemas.

Answer: B

63) When two subagents at quicktechie.com provide conflicting statistical data from different credible sources during a synthesis task, what is the architecturally correct pattern for handling this uncertainty?

A. Arbitrarily select the value from the most recently updated source to resolve the conflict smoothly.

B. Silently average the conflicting numerical values to provide a balanced estimate to the user.

C. Preserve both values and explicitly annotate the conflict with source attribution, letting the coordinator or user reconcile it.

D. Drop the contested data point entirely to ensure the final report only contains unified facts.

Answer: C

www.shapingpixel.com

64) A developer at quicktechie.com is building a highly experimental MCP server testing a new vector database. They want to use it locally in Claude Code without exposing the configuration to the rest of the team. Which configuration file should they use?

A. .mcp.json at the project root

B. ~/.claude.json

C. .claude/rules/mcp.md

D. mcp_local.config

Answer: B

65) When designing JSON schemas and interfaces for an agent's tools, which of the following practices represent recommended tool design principles? Choose 2 correct answers.

A. Keeping tool descriptions under 10 words to minimize context window token consumption.

B. Expanding tool descriptions to explicitly document what happens with empty inputs or invalid boundary data.

C. Splitting a generic document analysis tool into distinct, purpose-built extract_data and verify _ claim tools.

D. Combining all related backend API endpoints into a single monolithic tool to simplify the allowedTools array.

Answer: B,C

66) In an automated code review workflow, why is it considered a major architectural anti-pattern to have the same Claude session both generate code and subsequently review its own generated changes?

A. It exceeds the maximum token limits for the context window during the generation phase.

B. The reviewer retains the reasoning context and assumptions from the generation phase, creating confirmation bias.

C. The Message Batches API does not support multi-turn tool calling within a single request.

D. Tools invoked during the first pass are permanently locked and cannot be reused in the review pass.

Answer: B

www.shapingpixel.com

67) When using Claude Code for test generation and feature implementation, what is the recommended Test-Driven Development (TDD) iterative refinement pattern?

A. Run the /test slash command to automatically infer and generate missing unit tests for the entire project.

B. Write implementation write tests -> prompt Claude to 'make it better' -> refactor.

C. Write a failing test -> implement code to pass the test -> run tests -> refine implementation while keeping tests green.

D. Use Plan Mode to write all tests simultaneously, then execute them in a single batch.

Answer: C

68) Why is it heavily recommended to use Agent SDK hooks rather than prompt instructions to enforce critical business rules in a application?

A. Hooks inherently consume zero tokens from the context window.

B. Hooks provide deterministic guarantees, whereas prompt instructions rely on probabilistic compliance and carry a non-zero failure rate.

C. Hooks automatically implement strict JSON schema validation for all tool_use calls.

D. Prompt instructions cannot be modified without restarting the entire MCP server architecture.

Answer: B

69) In which scenario is it architecturally more reliable to start a fresh session injected with a structured summary rather than using - - resume to continue an existing session?

A. When the prior tool results in the existing session's context window have become completely stale.

B. When you want to maintain the exact reasoning chain from the previous generation steps.

C. When the target codebase has not undergone any structural changes.

D. When executing a fast, single-file typo correction.

Answer: A

www.shapingspixel.com

70) What is the primary architectural difference between handling a transient error and handling a valid empty result within an MCP tool's response payload?

A. Transient errors use isError: true with a specific errorCategory, while empty results use isError: false and return an empty data array.

B. Transient errors must bypass the MCP client and communicate directly via WebSockets, whereas empty results use standard HTTP.

C. Transient errors require the agent to spawn a subagent for resolution, whereas empty results are handled in the main thread.

D. There is no difference; both scenarios should return a generic HTTP 404 response.

Answer: A

71) What is a primary architectural limitation of using a fixed prompt chaining pipeline for a complex debugging task?

A. It cannot dynamically adapt its approach if an intermediate analysis step reveals an unexpected root cause.

B. It inherently exceeds the maximum token limit of the context window faster than single prompts.

C. It requires the use of the Message Batches API, adding a 24-hour latency to the execution.

D. It forces the agent to use tool_choice: any, bypassing deterministic programmatic hooks.

Answer: A

72) When processing a batch of 10,000 documents via the Message Batches API, 45 requests fail due to context window limits. What is the recommended failure handling strategy?

A. Resubmit the entire batch of 10,000 requests to guarantee correct ordering.

B. Resubmit only the failed documents (identified by their custom_id) after modifying them, such as chunking oversized documents.

C. Switch immediately to the synchronous API to retry the failed documents.

D. Ignore the failures, as the API inherently guarantees a 99% success SLA.

Answer: B

www.shapingpixel.com

73) What is the recommended architectural pattern to prevent the loss of critical transactional details (like account IDs, exact amounts, and dates) during long-running interactions?

A. Use the /compact command automatically after every 10 turns.

B. Extract these facts into a persistent, immutable 'case facts' block positioned at the start of the context.

C. Progressively summarize the conversation history and rely on the model's extended thinking parameters.

D. Switch to a higher-tier model with a 1 million token context window.

Answer: B

74) How should subagents handle transient failures before propagating errors to the coordinator?

A. Implement local error recovery within the subagent and only propagate errors that cannot be resolved locally.

B. Immediately propagate all failures to ensure the coordinator maintains full observability.

C. Silently drop the failed query and proceed with other data sources without notifying the coordinator.

D. Use a PreToolUse hook to block the error from reaching the coordinator entirely.

Answer: A

75) When using the Claude API, how must a system prompt be formatted to successfully utilize prompt caching?

A. Provide the system prompt as a standard string and set prompt_caching: true at the top level of the API request.

B. Structure the system prompt as a dictionary with a text block and include a cache_control field set to {"type": " ephemeral " }

C. Include the system prompt inside an XML block labeled .

D. Use the Message Batches API to ensure the system prompt is automatically cached across all batched requests.

Answer: B

www.shapingpixel.com

76) How can an architect split a massive, monolithic CLAUDE. md file to selectively include external standards files based on the current module?

A. By utilizing the #include macro inside the file's YAML frontmatter.

B. By using the @import syntax to reference and pull in external markdown files.

C. By compiling a PreToolUse hook to merge files dynamically before generation.

D. By specifying an allowed-tools array pointing to specific directories.

Answer: B

77) To locate all test files written in TypeScript within a specific directory subtree (e.g., **/*. test. tsx), which built-in tool is architecturally correct to use?

A. Grep

B. Glob

C. Bash

D. Read

Answer: B

78) During an extended codebase exploration session in Claude Code, context degradation begins to occur. What is the most effective strategy to persist key findings across context boundaries?

A. Use the - -json-schema flag to aggressively compress output.

B. Have agents maintain scratchpad files to record key findings and reference them in subsequent questions.

C. Increase the model's temperature parameter to recall lost context.

D. Continuously re-feed the entire codebase into the prompt every 5 turns.

Answer: B

www.shapingpixel.com

79) A pull request modifies 14 files across the stock tracking module. A single-pass self-evaluation review analyzing all files together produces inconsistent results: detailed feedback for some files but superficial comments for others. How should you restructure the evaluation architecture?

A. Split into focused passes: analyze each file individually for local issues, then run a separate integration-focused pass examining cross-file data flow.

B. Require developers to manually split the PR into smaller submissions before the automated review runs.

C. Switch to a higher-tier model with a larger context window to give all 14 files adequate attention in one pass.

D. Run three independent review passes on the full PR and only flag issues that appear in at least two of the three runs.

Answer: A

80) The data extraction system needs to categorize document types using an enum schema field. However, new and unexpected document types occasionally appear in production. How should you design the schema to handle this ambiguity?

A. Use a rigid enum containing only known document types to force the model to pick the closest match.

B. Use an open-ended string field instead of an enum to maximize flexibility.

C. Include an 'other' enum value paired with a 'document_type_detail' string field, guided by 2-4 few-shot examples covering edge cases.

D. Deploy a PreToolUse hook to intercept and reject any document types not pre-configured in the schema.

Answer: C

81) To keep the main agent's context clean while investigating a complex authentication flow, which architectural pattern is highly recommended?

A. Spawn a subagent to isolate the verbose exploration output, returning only a focused summary to the main agent.

B. Use the Message Batches API to process the exploration synchronously.

C. Apply progressive summarization to the main conversation history every 2 turns.

D. Use the fork_session command to duplicate the main agent permanently.

Answer: A

www.shapingpixel.com

82) To ensure that a model consistently classifies the severity of identified code issues, what is the best prompt engineering technique?

A. Define explicit severity criteria with concrete code examples for each severity level.

B. Instruct the model to self-report a confidence score from 1-10 for each finding.

C. Allow the model to dynamically generate severity labels based on context.

D. Lower the max_tokens limit so the model focuses only on critical bugs.

Answer: A

83) When providing Claude with a 50-page document, you notice it accurately recalls details from the first and last pages but misses a critical instruction buried on page 25. What is this phenomenon called?

A. The "lost in the middle" effect

B. Context truncation

C. Progressive summarization

D. Attention dilution

Answer: A

84) Which of the following are valid iterative refinement techniques used to achieve progressive improvement in output quality? Choose 2 correct answers.

A. Providing specific test cases with example input and expected output to correct edge-case handling failures.

B. Using the Interview Pattern to have Claude surface design considerations before implementing.

C. Executing same-session self-reviews to retain reasoning context and confirmation bias.

D. Relying on the tool_choice: "any" parameter to automatically fix failed code logic.

Answer: A,B

www.shapingpixel.com

85) How should a quicktechie.com application handle verbose tool outputs (e.g., an order lookup returning 40+ fields when only 3 are relevant) to optimize the context window?

A. Pass the full output and rely on the model's semantic focus to ignore irrelevant fields.

B. Trim the tool outputs programmatically to include only the return-relevant fields before they accumulate in the context history.

C. Use a PreToolUse hook to block the API response.

D. Require the agent to run a web search to summarize the JSON object.

Answer: B

86) What is the primary difference in tool usage between Plan Mode and Direct Execution in Claude Code?

A. Plan Mode uses Write and Edit heavily to scaffold boilerplate code.

B. Plan Mode restricts Claude to using only read-only tools to analyze the codebase before executing.

C. Plan Mode completely disables all tool usage until the user explicitly permits it.

D. Plan Mode automatically switches to a higher-tier model with larger tool capabilities.

Answer: B

87) Which of the following patterns represent best practices for implementing handoffs and enforcement in multi-step workflows? Choose 2 correct answers.

A. Using prompt-based instructions to guarantee that a lookup_order tool is always called before

B. process _ refund.

C. Compiling structured handoff summaries containing root causes and recommended actions when escalating to human agents.

D. Implementing programmatic prerequisites that explicitly block downstream tool calls until required verification steps are completed.

E. Relying on the agent's self-reported confidence score to trigger mid-process escalation decisions.

Answer: B,C

www.shapingpixel.com

88) According to best practices, which of the following scenarios should trigger a developer to use Plan Mode in Claude Code? Choose 2 correct answers.

A. When implementing multi-file architectural changes.

B. When adding a single console. log statement to a known file.

C. When evaluating multiple valid approaches for a new feature implementation.

D. When fixing a minor typo in project documentation.

Answer: A,C

89) In a multi-agent system, a subagent times out querying a database. How should this error be propagated back to the coordinator agent?

A. Catch the timeout and return an empty result set marked as successful to keep the workflow moving.

B. Return structured error context including the failure type, what was attempted, and partial results.

C. Propagate the exception directly to a top-level handler to terminate the entire workflow.

D. Return a generic string {"error": "Operation failed"}.

Answer: B

90) When designing JSON schemas for an extraction tool, which of the following practices are highly recommended for reliability? Choose 2 correct answers.

A. Setting tool_choice: "auto" to guarantee structural compliance across all documents.

B. Defining fields as optional or nullable (e.g., "["string "null"]) to prevent fabrication of missing data.

C. Including an other enum value alongside a detail string field to handle ambiguous or extensible categories.

D. Omitting the required array entirely to give the model maximum flexibility.

Answer: B,C

www.shapingpixel.com

91) An agent queries multiple external APIs that return dates in drastically different formats (e.g., Unix timestamps vs ISO 8601). What is the architecturally correct way to normalize this data before the agent processes it?

A. Use a PreToolUse hook to dynamically modify the prompt, instructing the agent to convert all dates to ISO 8601.

B. Implement a PostToolUse hook that intercepts the heterogeneous tool results and transforms them into a standardized format before appending to context.

C. Set the tool_choice parameter to strictly enforce ISO 8601 schema outputs.

D. Provide few-shot examples in the system prompt showing the agent successfully translating Unix timestamps.

Answer: B

92) You are tasked with using Claude Code to add comprehensive unit tests to an unfamiliar legacy codebase at quicktechie.com. Which approach represents the recommended decomposition strategy for this open-ended task?

A. Start by writing tests for the first file in the directory structure and proceed sequentially through the codebase.

B. Request Claude to automatically generate a monolithic test suite in a single prompt execution.

C. First map the project structure, identify high-impact areas, and create a prioritized plan that adapts as dependencies are discovered.

D. Use a fixed prompt chaining pipeline that executes npm test after generating each line of code.

Answer: C

93) In a coordinator-subagent research system, a web search subagent times out while attempting to retrieve information on a complex topic. How should this failure be propagated back to the coordinator agent?

A. The subagent should catch the timeout and return an empty JSON array L] marked as successful so the workflow continues smoothly.

B. Propagate the timeout exception directly to a top-level handler to immediately terminate the entire research workflow.

C. Implement automatic retry logic indefinitely within the subagent, returning a generic search unavailable status if it ultimately fails.

D. Return structured error context to the coordinator, including the failure type, the attempted query, any partial results, and potential alternative approaches.

Answer: D

94) When implementing multi-turn conversations with tool usage in the Claude API, how must the application handle the follow-up request after executing a tool?

A. Pass the tool result inside an assistant message with an is_error flag.

B. Send only the new tool result to save token context space in the context window.

C. Append a user message containing a tool_result block with the matching tool_use_id, and include the full prior conversation history.

D. Use the /compact command to summarize the tool output before returning it.

Answer: C

95) A developer at quicktechie.com is assigned a task to migrate a core utility library that is imported by over 45 different files across the repository. Which execution approach should the developer take?

A. Use Direct Execution to make incremental changes and let the implementation naturally reveal dependency boundaries.

B. Use Plan Mode to explore the codebase, understand dependencies, and design an architectural approach before committing to file modifications.

C. Use a custom slash command designed to bypass context limits.

D. Start with Direct Execution, and manually switch to Plan Mode only if unexpected compilation errors occur.

Answer: B

96) An MCP tool configured for a quicktechie.com application fails to connect to its backend database due to a timeout. Why is it considered a critical anti-pattern to silently return an empty JSON array [] for this failure?

A. It immediately triggers a fatal exception that terminates the entire agentic loop.

B. It misleads the agent into believing the search was successfully executed but yielded zero matches, causing catastrophic misunderstandings.

C. It forces the Claude API to throw a schema validation error.

D. It automatically triggers an infinite retry loop that consumes unnecessary tokens.

Answer: B

www.shapingpixel.com

97) Using the FastMCP Python SDK, which decorators are used to expose capabilities to an MCP client? Choose 2 correct answers.

A. @mcp.tool

B. @mcp.agent

C. @mcp.action

D. @mcp.resource

Answer: A,D

98) To extract metadata from a highly variable set of invoices, you use an enum field for document_type. How should you handle unexpected document types to maintain extraction quality?

A. Include an other enum value paired with a detail string field for extensible categorization.

B. Omit the enum constraint entirely and use an open-ended string field.

C. Implement a PreToolUse hook to drop unrecognized document types.

D. Provide an enum containing only known types and rely on the model to find the closest match.

Answer: A

99) Why is configuring the exact same Claude session to review its newly generated pull request code considered an architectural anti- pattern?

A. It exceeds the maximum token limit for the context window during the generation phase.

B. The reviewer retains the reasoning context and assumptions from the generation phase, creating confirmation bias.

C. The Message Batches API does not support multi-turn tool calling within a single request.

D. Tools invoked during the first pass are permanently locked and cannot be reused in the review pass.

Answer: B

www.shapingpixel.com

100) What is the primary architectural difference between enforcing a business rule via a system prompt versus using an Agent SDK hook?

A. Prompts execute after the tool runs, while hooks execute before the prompt is processed.

B. Prompts are deterministic, while hooks rely on the model's probabilistic reasoning.

C. Hooks provide 100% reliable deterministic enforcement, whereas prompts are probabilistic and can occasionally be ignored by the LLM.

D. Prompts require an active MCP server connection, while hooks run strictly within the Claude desktop application.

Answer: C

www.shapingpixel.com

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely.

What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- ○ A. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity
- ◉ B. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis
- ○ C. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations
- ○ D. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

You've configured the system so that all four subagents have access to the complete set of 18 tools. During testing, agents frequently call tools outside their specialization—the synthesis agent attempts web searches, and the report generator tries to analyze documents.

What is the primary cause of this poor tool selection behavior?

- ○ A. The coordinator cannot track which capabilities each subagent has, leading to misrouted tasks.
- ○ B. The tool definitions consume too much context window space, leaving insufficient room for task content.
- ◉ C. Choosing from 18 tools instead of 4-5 relevant ones increases decision complexity beyond reliable selection thresholds.
- ○ D. The agents' role descriptions in their system prompts conflict with having access to tools outside that role.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow.

What change would most improve briefing quality?

- ○ A. Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation
- ○ B. Standardize all subagent outputs to JSON with fields for `claim`, `evidence`, `source`, and `confidence`
- ◉ C. Update the synthesis agent to render each content type appropriately—financial data as tables, news as prose
- ○ D. Standardize all subagent outputs to prose summaries with inline citations

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations.

What context-passing strategy provides the best balance of completeness and efficiency?

- ◉ A. Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.
- ○ B. Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.
- ○ C. Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.
- ○ D. Pass the full accumulated context from all prior agents.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research.

What's the most effective way to improve response time for these follow-up summaries?

- ○ A. Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.
- ◉ B. Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis.
- ○ C. Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.
- ○ D. Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate $50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates $35B (±$7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market may be $35B-$50B depending on factors."

What systematic approach best addresses this?

- ○ A. Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.
- ○ B. Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0-1.0), then weight-average findings by their calibrated confidence.
- ○ C. Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator.
- ◉ D. Instruct the synthesis agent to structure reports with explicit sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The synthesis agent completes its initial pass but flags that three key research questions remain unanswered because the web search and document analysis agents didn't find relevant information on those specific subtopics. The coordinator currently proceeds directly to report generation, producing reports with incomplete coverage.

What change would most effectively improve research completeness?

- [x] A. Have the coordinator evaluate synthesis output for gaps, then re-delegate to web search and document analysis with targeted queries before invoking synthesis again.
- [ ] B. Have the report generation agent note which research questions couldn't be answered, so users understand the limitations of the final output.
- [ ] C. Give the synthesis agent direct access to web search tools so it can autonomously fill knowledge gaps without returning control to the coordinator.
- [ ] D. Increase the initial breadth of queries sent to web search and document analysis to reduce the probability of missing relevant information.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Production monitoring shows the research phase takes longer than expected. Analysis reveals the coordinator invokes the web search subagent, waits for its response, then invokes the document analysis subagent and waits again. These tasks are independent—neither requires the other's output.

How should you modify the system to run these subagents concurrently?

- [ ] A. Create an async orchestration layer outside the agent that spawns parallel threads, each running a separate coordinator-subagent pair, then aggregates results.
- [ ] B. Switch both subagents to use a Haiku-tier model instead of Sonnet to reduce their individual execution time.
- [ ] C. Add detailed instructions to the coordinator's system prompt explaining the performance benefits of parallel execution and requesting it invoke both subagents at the same time.
- [x] D. Structure the coordinator to emit both Task tool calls (for web search and document analysis) in a single response message rather than across separate conversation turns.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The coordinator agent has AgentDefinitions configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like "I'll ask the web search agent to find sources on this topic"—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors.

What is the most likely cause?

- A. The coordinator's max_tokens setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.
- B. The AgentDefinitions are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.
- C. Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in ClaudeAgentOptions.
- **D. The coordinator's allowedTools configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.**

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

After the web search and document analysis subagents complete their tasks, the coordinator needs to spawn the synthesis subagent to synthesize the findings.

What is the correct approach for providing the synthesis subagent with the information it needs?

- A. Spawn the subagent with only a brief task description, relying on automatic context inheritance from the coordinator
- B. Provide the subagent with tool definitions that allow it to request outputs from other subagents via callbacks
- **C. Include the complete findings from both subagents directly in the synthesis subagent's prompt**
- D. Pass reference identifiers and configure the subagent with read access to a shared memory store where other subagents deposited their results

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analyzer had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings.

What state management approach best balances information fidelity with context efficiency when restoring agent state?

- (●) A. Have each agent persist a structured export to a known location. On resume, the coordinator loads the manifest and injects relevant state into agent prompts.
- ( ) B. Have each agent maintain its own persistent state file and reload it independently at the start of each session.
- ( ) C. Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.
- ( ) D. Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id.

What's the most cost-effective approach to process these failures?

- ( ) A. Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch
- ( ) B. Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts
- (●) C. Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions
- ( ) D. Resubmit the entire 10,000 document batch using a model tier with a larger context window

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for attendee_count when the source contains no attendance information.

What's the most effective way to reduce these false extractions?

- ( ) A. Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.
- (●) B. Add prompt instructions to return null for any field where information is not directly stated in the source.
- ( ) C. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.
- ( ) D. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction uses tool use with a JSON schema where `property_type` is defined as an enum: `['house', 'apartment', 'condo', 'townhouse']`. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types—"studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse"—and new types continue appearing regularly.

What's the most effective long-term solution?

- (●) A. Add an "other" value to your enum with a separate `property_type_detail` string field for specifics when "other" is selected.
- ( ) B. Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.
- ( ) C. Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.
- ( ) D. Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence ≥90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions.

Before deploying, what validation step is most critical?

- (●) A. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.
- ( ) B. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.
- ( ) C. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.
- ( ) D. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions (≥85%) also contain errors—cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time.

What approach is most effective?

- A. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.
- B. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.
- C. Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.
- D. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline occasionally receives responses that don't parse as valid JSON, causing downstream processing failures. The current implementation prompts Claude to return JSON in the response text and then parses it.

What is the most reliable approach to ensure Claude returns valid, schema-compliant structured data?

- A. Use regular expressions to locate and extract JSON from the response text, handling cases where Claude includes explanatory text around the JSON block.
- B. Add explicit formatting instructions to the prompt with JSON examples, emphasizing that Claude must return only valid JSON with no surrounding text.
- C. Define a tool with a JSON schema specifying the expected structure, using tool_use to constrain Claude's output to schema-compliant JSON.
- D. Implement a retry loop that catches JSON parse errors and re-prompts Claude with the error details, asking it to correct the malformed output.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After deployment, you find that 12% of extractions contain semantic errors that pass JSON schema validation (e.g., a duration like "30 minutes" incorrectly placed in an ingredient quantity field). Human reviewers have capacity to check only 20% of extractions.

Which approach most effectively allocates reviewer attention?

- A. Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.
- B. Prioritize review of all extractions where required fields are empty or explicitly marked as not found.
- C. Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set.
- D. Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned."

What's the most effective improvement?

- [x] A. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- [ ] B. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."
- [ ] C. Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.
- [ ] D. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide.

What's the most effective way to ensure structured metadata extraction happens first?

- [ ] A. Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- [ ] B. Set `tool_choice` to `"auto"` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- [x] C. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- [ ] D. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your invoice extraction uses tool use with strict JSON schemas. JSON syntax errors never occur, but 12% of extractions fail semantic validation—for example, line item amounts don't sum to the extracted total, or vendor IDs don't match valid formats. These failures currently route to manual review.

What's the most effective approach to reduce manual review volume while maintaining accuracy?

- A. Implement post-processing logic that automatically corrects common errors, such as recalculating totals from line items when sums don't match.
- B. When validation fails, make a follow-up request with the document, extraction, and validation errors for model correction.
- C. Retry the extraction up to 3 times when validation fails, accepting the first result that passes validation.
- D. Add stricter schema constraints with detailed field descriptions to prevent the model from generating invalid values initially.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field—sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source.

What's the most effective way to improve extraction consistency?

- A. Set temperature to 0 to eliminate randomness and ensure deterministic outputs
- B. Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability
- C. Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats
- D. Make the "materials" field required instead of optional in the schema to force the model to always extract a value

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system uses tool_use with a JSON schema containing 12 fields and detailed descriptions, totaling approximately 2,500 tokens for the complete tool definition. Processing documents under 150K tokens yields 98% accuracy. For documents between 175-190K tokens, accuracy drops to 71%, with information from the final third consistently missed. The model's context window is 200K tokens.

What is the most likely cause?

- A. Schemas exceeding 8-10 fields increase decision complexity during parameter generation, reducing extraction accuracy independent of document length.
- B. The model distributes attention proportionally across input length, causing fields mentioned only once near the document's end to receive insufficient processing focus.
- C. Very long documents exceed the model's effective attention span regardless of context limits, causing accuracy degradation for content farther from the prompt instructions.
- D. Tool definitions consume input context tokens. Combined with system prompts and document content, the total approaches the context limit, degrading end-of-document processing.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON.

What approach provides the most reliable schema compliance?

- A. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.
- B. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.
- C. Define a tool with your target schema as input parameters and have Claude call it with the extracted data.
- D. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements.

How should you architect the processing pipeline?

- A. Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- B. Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- C. Submit all documents to the Batch API with custom_ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.
- D. Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats—inline citations vs. bibliographies, methodology sections vs. details embedded in introductions.

What's the most effective way to address these failures?

- A. Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.
- B. Add few-shot examples demonstrating extractions from documents with varied structures—showing how to identify citations in different formats and locate methodology details across section types.
- C. Implement retry logic that re-sends requests when validation detects empty required fields.
- D. Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

A developer uses Claude Code to refactor a function during their development session. Before committing, they ask the same Claude session to review the code for issues. Later, a separate automated CI review catches several bugs that the same-session review missed.

What best explains this discrepancy?

- A. The CI environment has access to the full codebase context while the local session only sees the current file
- B. Claude retains context about its prior reasoning in the session, making it less likely to question its own decisions
- C. The extended session length caused the context window to fill with conversation history, leaving less room for thorough analysis
- D. The CI review uses a more specific prompt tailored for catching bugs, while the developer's request was too general

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your CI pipeline performs security-focused code reviews on approximately 50 PRs daily, currently costing $150/day using the synchronous API. Reviews are non-blocking—developers merge after tests pass and address findings in follow-up commits. You're evaluating the Message Batches API for its 50% cost reduction.

What factor most determines whether batch processing is appropriate for this use case?

- ○ A. Whether your result processing can handle reviews arriving in a different order than submitted.
- ○ B. Whether reducing per-review latency from 30-60 seconds to near-instant matters for your workflow.
- ◉ C. Whether review feedback arriving up to 24 hours after PR creation remains actionable.
- ○ D. Whether you can structure each review as a single request without multi-turn refinement.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your test generation produces unit tests for new code, but reviews show 55% are low-value: trivial assertions that only verify functions don't throw exceptions, tests duplicating existing coverage, or tests ignoring your team's fixture conventions.

How do you reduce the rate of low-value tests being generated in the first place?

- ◉ A. Document testing standards in CLAUDE.md including valuable test criteria, available fixtures with intended use cases, and examples distinguishing meaningful behavioral tests from trivial assertions.
- ○ B. Add post-generation coverage analysis that automatically filters out any generated test that doesn't increase line coverage beyond what existing tests provide.
- ○ C. Implement a two-phase generation where a second Claude call scores each test against quality criteria, filtering out low-scoring tests before presenting results to developers.
- ○ D. Restrict test generation to directories where historical quality metrics show higher acceptance rates, disabling it for areas where generated tests consistently require heavy editing.

You are integrating Claude Code into your Continuous ntegration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable eedback and minimize false positives.

Your automated code review is missing genuine bugs in pull requests. Investigation reveals that your review prompt includes the instruction: "Only flag critical issues that would definitely cause production failures. Ignore minor concerns and anything you're uncertain about." Developers confirm that some missed bugs are real logic errors the model investigated but chose not to report. The team requires that review output remain structured (each finding tagged with metadata) and actionable.

Which prompt change both removes the cause of the suppressed findings and preserves structured, tagged output for downstream filtering?

- ○ A. Add a second review pass that re-reads the diff using the same prompt, looking for anything the first pass may have missed.
- ○ B. Enable extended thinking and instruct the model to reason step-by-step about each code change before producing its review.
- ○ C. Remove all severity-related instructions from the prompt and let the model use its default judgment about what to report.
- ◉ D. Instruct the model to report all findings with a confidence level and severity tag, deferring filtering to a downstream step.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated review calls the Claude API for each PR, using tool_use with a `report_findings` tool that returns a JSON array of finding objects (each with `file_path`, `line_number`, `severity`, `category`, and `description`). During testing on a large PR touching 30+ files, the response hits the `max_tokens` limit and the output is truncated mid-JSON, causing your pipeline's parser to fail.

What is the most effective way to handle this?

- ◉ A. Split the review into multiple API calls that each analyze a subset of the changed files, then merge the resulting findings arrays.
- ○ B. Increase `max_tokens` to the model's maximum and instruct Claude to keep finding descriptions under 50 words each.
- ○ C. Add retry logic that detects truncated JSON and re-sends the request with instructions to report only critical and high severity findings.
- ○ D. Switch from tool_use to prompting Claude to return findings as a markdown list.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated review CI jobs take 18 seconds to initialize before Claude begins analyzing code. Profiling reveals the delay comes from auto-discovery of hooks, MCP servers, plugins, skills, and multiple nested CLAUDE.md files throughout your monorepo. You need to cut startup time while ensuring reviews still enforce your team's coding standards, which are documented in your root-level CLAUDE.md file.

What is the most effective approach?

- A. Replace the default prompt entirely using --system-prompt-file ./CLAUDE.md, which bypasses default prompt assembly and loads only your project rules.
- B. Run with --bare mode and pass --append-system-prompt-file ./CLAUDE.md to explicitly load your project standards while skipping all auto-discovery.
- C. Keep the default initialization and add --exclude-dynamic-system-prompt-sections to reduce per-machine prompt variability and improve prompt cache hit rates across runners.
- D. Run with --bare mode and specify all review criteria directly in the -p prompt argument for each CI invocation, without referencing any external files.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your automated reviewer uses a single prompt covering security issues, API design, and business logic correctness. Your evaluation suite shows strong recall for API design findings (82%) but poor recall for business logic edge cases in quiz scoring (34%). When you add few-shot examples of logic bugs to the prompt, logic recall improves to 41% but API design recall drops to 68%.

How should you address this trade-off to improve detection across both categories?

- A. Split the review into separate focused prompts — one for security and API design, another for business logic — each with dedicated examples, then consolidate findings before posting.
- B. Upgrade to a more capable model tier, since its stronger reasoning will handle both concern types in a single prompt and eliminate the recall trade-off.
- C. Replace the few-shot examples with a detailed checklist of specific logic edge cases to verify, such as division-by-zero in score calculations and boundary conditions in grading thresholds.
- D. Provide the full repository as context instead of just the changed files and surrounding code, giving the model deeper visibility into business logic patterns.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

After deploying automated code review, developers report that approximately 35% of flagged findings are false positives falling into consistent patterns: style suggestions contradicting team conventions, security warnings for patterns safe in your deployment context, and performance suggestions that would degrade your specific use case. You want to reduce false positives while maintaining the ability to catch genuine issues.

Which approach best enables the model to generalize its judgment to novel code patterns it hasn't seen before?

- A. Add instructions to your system prompt to "be conservative," "only flag definite issues," and "consider that some patterns may be intentional."
- B. Include few-shot examples in your prompt showing annotated code snippets that distinguish acceptable patterns from genuine issues in each category.
- C. Implement post-processing that uses keyword matching to filter out findings containing terms like "convention," "context-dependent," or "trade-off."
- D. Create a comprehensive written specification of all patterns that should not be flagged, then include this full documentation in the system prompt.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline includes a release notes generation step that classifies and summarizes approximately 200 commits at the end of each weekly release cycle. Each commit is currently sent as a separate Messages API call using a Sonnet-tier Claude model. The release notes aren't needed until the following morning (results have ~12 hours of acceptable latency). Your team needs to reduce per-token API cost for this step while keeping the same model and prompts (no change to model tier or output quality).

Which approach satisfies all of these constraints?

- A. Submit the 200 requests to the Message Batches API with unique custom_ids and retrieve results once the batch ends, which applies a 50% discount to all input and output tokens.
- B. Issue the 200 Messages API requests in parallel using concurrent connections, since concurrency lowers the per-token price charged by the API.
- C. Switch the summarization calls from the Sonnet-tier model to a Haiku-tier model to take advantage of Haiku's lower per-token rates.
- D. Concatenate all 200 commit messages into a single Messages API request and have the model return all summaries in one response, since fewer requests always reduces total token cost.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

The automated review consistently flags patterns your team uses intentionally — force-unwrapping optionals in test files, using large coordinator classes that follow your established architecture, and importing internally-maintained modules marked as deprecated in the public SDK. Developers are dismissing roughly 30% of all findings as project-specific false positives.

Which approach prevents the model from generating these findings in the first place by supplying the project's conventions as persistent context on every review?

- A. Build post-processing keyword filters that suppress findings containing terms like "force unwrap," "large class," or "deprecated import" before results reach developers.
- B. Document the team's accepted patterns and intentional conventions in the project's CLAUDE.md file so the model receives this context during every review.
- C. Have developers add inline suppress-comments at flagged lines and preprocess diffs to exclude suppressed lines before sending code to the model.
- D. Configure the review to analyze only the changed lines in the diff without surrounding file context, reducing the amount of code the model evaluates per review.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Your pipeline reviews every PR using a single API call with a static prompt containing the diff and full text of each changed file — unchanged files are not included. Reviews are posted asynchronously and don't block PR creation. Developers report that reviews consistently miss bugs involving cross-file interactions — for example, a PR renames a function's parameters but the review doesn't flag callers in unchanged files that still use the old argument order. Evaluation shows cross-file bugs account for 35% of production incidents from reviewed PRs.

What is the most effective change to your review design?

- A. Use static analysis to build a dependency graph of changed code, then expand the prompt to include all files within two dependency hops of any changed file.
- B. Redesign the review as a turn-limited agentic task where the model can read files and search the codebase via tools, following references to verify cross-file findings.
- C. Run parallel review passes per changed file with direct dependents included in each pass, then aggregate and deduplicate findings using a final summarization call.
- D. Add chain-of-thought instructions asking the model to list all external references in the diff, then reason step-by-step about how each change might affect callers in other files.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

enforced by your linter in CI, (2) findings on auto-generated template code under `src/gen/`, and (3) rendering helper patterns that are intentional project conventions but get flagged because they resemble common anti-patterns. Only about 4 findings per PR are genuine logic bugs.

What is the most effective way to reduce this noise while preserving detection of real issues?

- A. Configure separate GitHub Action workflow files for each code area, one for generated code with findings suppressed, one for rendering code with custom review instructions, and a general workflow for everything else.
- B. Add custom review instructions to a GitHub Actions workflow file, using the action's prompt parameter to suppress lint-duplicate findings, ignore generated template code, and apply stricter evidence requirements for rendering-related issues.
- C. Add detailed explanations to the project's CLAUDE.md describing which patterns are intentional, that linting is handled separately by CI, and that the src/gen/ directory contains auto-generated template code.
- D. Create a REVIEW.md at the repository root with skip rules for CI-enforced checks and generated files, and a verification requirement that rendering-related findings must cite a specific line demonstrating incorrect behavior.

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

PROMPT="$PROMPT and style violations." claude -p \ --dangerously-skip-permissions \ --system-prompt "$PROMPT" \ < diff.txt ``` The reviews complete and return feedback, but Claude only comments on the piped diff text — it never reads surrounding files in the checked-out repository to understand broader context, even when the diff modifies a function called by many other modules.

Which change to the invocation will cause Claude to read related files in the repository while still applying your custom review instructions?

- A. Stop piping the diff via stdin and instead embed the diff contents inside the prompt string, so Claude Code treats the invocation as an agentic session rather than a stream-processing one.
- B. Replace `--system-prompt` with `--append-system-prompt` so your review instructions are added to Claude Code's default prompt instead of overwriting the built-in guidance for using file-reading and code navigation tools.
- C. Remove `--system-prompt` entirely and place the review instructions in a `CLAUDE.md` file at the repo root, since `--system-prompt` is incompatible with tool use under `-p`.
- D. Keep `--system-prompt` and add `--allowedTools "Read,Glob,Grep"` so that the non-interactive `-p` mode permits file system tools that it otherwise disables.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to find all files in the monorepo that import the `@company/auth` package to understand how authentication is used across services.

Which built-in tool is most appropriate for this task?

- A. Bash, to execute `find . -type d -name "*auth*"` and explore matching directories
- B. Read, starting with package.json files to trace dependency declarations
- C. Glob, to find files with "auth" in their filename or path
- D. Grep, to search for the import statement pattern across file contents

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search.

When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- A. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- B. The agent automatically selects the most relevant server based on the request and loads only that server's tools.
- C. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.
- D. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your productivity agent connects to three MCP servers: an issue tracker (search_issues, get_issue, create_comment), a documentation wiki (search_docs, get_page, list_spaces), and a database explorer (run_query, get_schema, list_databases). When engineers ask cross-system questions like "What database tables are affected by the authentication refactor in PROJ-1234?", monitoring shows the agent makes 8-10 sequential tool calls, frequently issues exploratory calls because it lacks visibility into what content each server contains, and exhausts context space before completing complex investigations.

What architectural change best leverages MCP capabilities to address these issues?

- [x] A. Expose each server's content catalog as MCP resources—issue summaries, documentation hierarchy, database schemas
- [ ] B. Add a prepare_investigation tool to each server that accepts a natural language question and returns relevant content summaries
- [ ] C. Consolidate all three servers into a unified MCP server with cross-referencing capabilities
- [ ] D. Add an orchestrator that routes questions to a single server based on keywords

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its old_string parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns.

What's the most reliable way to complete this insertion?

- [x] A. Use Read to load the file, add the function at the appropriate location, then Write the updated file
- [ ] B. Use Bash to append the function definition to the end of the file using heredoc syntax
- [ ] C. Use Edit's replace_all parameter to target a common pattern and embed the new function in the replacement text
- [ ] D. Use Edit with an extremely long old_string capturing 30+ lines of context to guarantee uniqueness

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You're building a security scanning workflow.

When engineers need to locate all occurrences of a dangerous function like eval() across a large codebase, which tool should your agent use for content search?

- A. Read the project's main entry file and follow import statements to trace where eval might be used.
- B. Use Grep to search for the pattern "eval(" across all files in the codebase.
- C. Use Bash to run `ls -R | grep eval` to recursively list files containing eval.
- D. Use Glob with a pattern like `**/*eval*` to find files, then Read each matching file.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

After adding an MCP server with specialized code refactoring tools (`extract_function`, `rename_variable`, `inline_function`), you notice the agent still uses basic text manipulation via Write and Bash sed commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like "extract_function: Extracts a function from code."

What's the most effective way to improve adoption of the MCP refactoring tools?

- A. Remove the Write tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.
- B. Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.
- C. Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.
- D. Accept this as expected behavior since simpler tools like sed are more predictable than specialized refactoring tools.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Engineers frequently ask the agent to cross-reference code changes with Jira tickets during reviews—checking ticket descriptions, acceptance criteria, and recent comments. This currently requires manually copy-pasting content into conversations. The team wants the agent to access this standard Jira ticket data directly.

What's the most effective approach?

- A. Use the Bash tool with `curl` to call Jira's REST API, including authentication headers and parsing JSON responses inline.
- B. Export Jira tickets to markdown files in the repository that the agent accesses using the Read tool.
- C. Build a custom MCP server wrapping Jira's API with tools designed specifically for this team's code review workflow.
- D. Integrate an existing Jira MCP server that exposes tickets, comments, and metadata through discoverable tool interfaces.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module).

What exploration strategy will most reliably identify all callers?

- A. Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.
- B. Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.
- C. Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- D. Use Grep to search for the function's original name across the codebase.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific `VulkanPipeline` and `FrameGraph` classes it discovered earlier.

What's the most effective approach?

- A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- B. Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.
- C. Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.
- D. Continue in the current context with more targeted prompts referencing the specific classes by name.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored.

What's the most effective approach to address this?

- A. Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.
- B. Create summaries of all source files before exploration begins, loading only these compressed representations into context.
- C. Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.
- D. Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code.

What exploration approach will most efficiently find the responsible code?

- (●) A. Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.
- ( ) B. Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- ( ) C. Use Glob to find files in directories commonly associated with error handling (such as `errors/`, `exceptions/`, or `handlers/`) across all services, then Read each matching file.
- ( ) D. Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.

Which task decomposition pattern is most appropriate for this workflow?

- (●) A. Prompt chaining—break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.
- ( ) B. Routing—classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.
- ( ) C. Single comprehensive prompt—include all instructions in one prompt and let the model handle all three aspects simultaneously.
- ( ) D. Orchestrator-workers—have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total).

What's the most effective next step for building understanding while managing context constraints?

- A. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.
- B. Use Glob to find files matching common caching patterns (*cache*.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.
- C. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- D. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration.

What's the most effective approach?

- A. Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid.
- B. Launch a fresh subagent with a summary of prior findings.
- C. Launch a fresh subagent and include the prior transcript in the initial prompt for context.
- D. Resume the subagent from its previous transcript and inform it about the renamed functions.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services.

What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- ○ A. Read all files containing "auth", "login", "permission", or "token" in their content or filename.
- ○ B. Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.
- ○ C. Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.
- ◉ D. Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The document analysis agent has a single `analyze_document` tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions.

What's the most effective way to improve reliability?

- ○ A. Keep the single tool but add an `analysis_type` enum parameter requiring explicit selection between extraction, summarization, and verification modes
- ○ B. Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent
- ◉ C. Split the generic tool into purpose-specific tools—`extract_data_points`, `summarize_content`, `verify_claim_against_source`—each with defined input/output contracts
- ○ D. Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications.

What's the most effective approach to optimize for varying query complexity?

- A. Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.
- B. Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.
- C. Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.
- D. Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations—the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps.

What's the most effective approach to ensure proper source attribution in the final reports?

- A. Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.
- B. Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.
- C. Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).
- D. Have the report generator query the web search agent to re-locate sources for claims in the final report.

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely.

What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- ○ A. Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity
- ◉ B. Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis
- ○ C. Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations
- ○ D. Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management, and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific `VulkanPipeline` and `FrameGraph` classes it discovered earlier.

What's the most effective approach?

- A. Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.
- B. Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.
- C. Use `/clear` to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.
- D. Continue in the current context with more targeted prompts referencing the specific classes by name.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth—having the agent propose specific code changes for each—before deciding which to implement.

What's the most effective way to structure this exploration?

- A. Use `fork_session` to create two branches from yesterday's analysis, exploring one approach in each fork.
- B. Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- C. Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.
- D. Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate—it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows.

What's the most effective approach to complete this investigation?

- A. Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.
- B. Spawn subagents to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level understanding.
- C. Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration.
- D. Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase has 200+ files and the developer doesn't know which components are involved. The agent must trace the error through routing, middleware, business logic, and database layers.

What task decomposition approach would be most effective?

- ○ A. Have the agent first create a comprehensive plan mapping all code paths through the endpoint before beginning any file exploration or code reading.
- ◉ B. Have the agent dynamically generate investigation subtasks based on what it discovers at each step, adapting its exploration plan as new information about the error path emerges.
- ○ C. Define a fixed sequence of investigation steps upfront —grep for error patterns, then read error handlers, then check database queries, then examine middleware—executing each step regardless of intermediate findings.
- ○ D. Run parallel worker agents that simultaneously investigate all four layers, then synthesize their findings to identify where the error originates.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

You've configured your Claude agent with three MCP servers: one for git operations, one for Jira ticket management, and one for documentation search.

When a user asks the agent to "create a branch for JIRA-123 and add documentation links to the ticket," how does the agent access tools across these servers?

- ◉ A. Tools from all configured MCP servers are discovered at connection time and available simultaneously to the agent.
- ○ B. You must specify which MCP server to use for each turn, and the agent can only access one server's tools at a time.
- ○ C. The agent queries each server sequentially to determine which handles each tool, routing calls based on tool name prefixes.
- ○ D. The agent automatically selects the most relevant server based on the request and loads only that server's tools.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

After integrating a local MCP server providing code analysis tools (`analyze_dependencies`, `find_dead_code`, `calculate_complexity`), you verify the server is healthy and tools appear in the `tools/list` response. However, you observe that the agent consistently uses Grep to search for import statements instead of calling `analyze_dependencies`—even when users explicitly ask about "code dependencies." Examining tool definitions reveals: - MCP: `analyze_dependencies` — "Analyzes dependency graph" - Built-in: Grep — "Search file contents for a pattern using regular expressions. Returns matching lines with line numbers and surrounding context."

What's the most effective approach to improve the agent's selection of MCP tools?

- ○ A. Add routing instructions to the system prompt specifying that dependency-related questions should use MCP tools rather than Grep.
- ○ B. Split `analyze_dependencies` into granular tools (`list_imports`, `resolve_transitive_deps`, `detect_circular_deps`) so each has a focused purpose less likely to overlap with Grep.
- ◉ C. Expand MCP tool descriptions to detail capabilities and outputs—e.g., "Builds dependency graph showing direct imports, transitive dependencies, and cycles."
- ○ D. Remove Grep from available tools when the MCP server is connected to eliminate functional overlap.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

1.5An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total).

What's the most effective next step for building understanding while managing context constraints?

- A. Use Glob to find files matching common caching patterns (*cache*.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.
- B. Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- C. Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.
- D. Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer submits two requests: - Request A: "Rename the `getUserData` function to `fetchUserProfile` everywhere it's used." - Request B: "Improve error handling throughout the data processing module—add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data."

For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

- A. Neither request benefits significantly
- B. Request B, the error handling task
- C. Both requests benefit equally
- D. Request A, the function rename task

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration.

What's the most effective approach?

- A. Launch a fresh subagent with a summary of prior findings.
- B. Resume the subagent from its previous transcript and inform it about the renamed functions.
- C. Launch a fresh subagent and include the prior transcript in the initial prompt for context.
- D. Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your code review assistant needs to analyze pull requests and provide feedback on three aspects: code style compliance, potential security issues, and documentation completeness. Each aspect requires reading files, running analysis tools, and generating a report section. The review process follows the same three-step workflow for every PR.

Which task decomposition pattern is most appropriate for this workflow?

- A. Orchestrator-workers—have a central LLM analyze each PR to dynamically determine which checks are needed, then delegate to specialized worker LLMs for each identified subtask.
- B. Routing—classify each PR by type (feature, bugfix, refactor) first, then route to different review prompts optimized for that category.
- C. Single comprehensive prompt—include all instructions in one prompt and let the model handle all three aspects simultaneously.
- D. Prompt chaining—break the review into sequential steps where each aspect (style, security, documentation) is analyzed separately, with outputs combined in a final synthesis step.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module).

What exploration strategy will most reliably identify all callers?

- A. Use Grep to search for the function's original name across the codebase.
- B. Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.
- C. Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- D. Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your agent has analyzed a complex service module—reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs.

How should you manage the sessions?

- A. Export the analysis session's key findings to a file, then create two new sessions that reference this file.
- B. Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- C. Start two fresh sessions, having each re-read the relevant source files before beginning.
- D. Resume the analysis session with fork_session enabled, creating a separate branch for each testing strategy.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer asks your agent to add comprehensive tests to a legacy codebase with 200 files and minimal existing test coverage. The engineer hasn't specified which modules to prioritize.

How should the agent decompose this open-ended task?

- (●) A. Use Glob and Grep to map codebase structure, identify heavily-coupled modules, create a prioritized plan for high-impact areas, and revise as dependencies are discovered.
- ( ) B. Create a fixed testing schedule upfront based on directory structure, allocating equal effort to each top-level directory regardless of code complexity or business importance.
- ( ) C. Start writing tests for the first module alphabetically, using test failures and imports to discover related files organically.
- ( ) D. Systematically read all 200 files to create a complete function inventory before writing any tests, ensuring the testing plan accounts for every function before beginning.

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

An engineer used Claude Code yesterday to investigate authentication flows in a legacy monolith, building up significant context over a 2-hour session. Today she wants to continue that specific investigation. She's worked on three other codebases since then and knows the session was named "auth-deep-dive".

How should she resume?

- ( ) A. Use `--session-id` with the UUID from yesterday's session transcript file
- ( ) B. Use `--continue` to pick up where the most recent conversation left off
- ( ) C. Start fresh and re-read the same files
- (●) D. Use `--resume auth-deep-dive` to load that specific session by name

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge.

What approach best balances efficiency and accuracy?

- ( ) A. Start a fresh session to ensure the agent works with current codebase state without stale assumptions
- ( ) B. Resume the session without informing the agent about the changed files
- (●) C. Resume the session and inform the agent which specific files changed for targeted re-analysis
- ( ) D. Resume the session and immediately have the agent re-read all 12 previously analyzed files

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge.

What approach best balances efficiency and accuracy?

- A. Start a fresh session to ensure the agent works with current codebase state without stale assumptions
- B. Resume the session without informing the agent about the changed files
- C. Resume the session and inform the agent which specific files changed for targeted re-analysis
- D. Resume the session and immediately have the agent re-read all 12 previously analyzed files

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide.

What's the most effective way to ensure structured metadata extraction happens first?

- A. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.
- B. Set `tool_choice` to `"auto"` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- C. Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- D. Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting—prices as "$12" vs "12.00", dietary info as icons vs text.

What's the most reliable approach?

- A. Request multiple extraction attempts per document and select the most common format.
- B. Use separate extraction calls for each field to ensure consistent handling of each type.
- C. Define a strict output schema and include format normalization rules in your prompt.
- D. Extract data as-is and normalize formats in post-processing code after Claude returns.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated "weight: 2.3 kg" in the extraction output.

What schema design change most effectively addresses this hallucination behavior?

- ○ A. Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text
- ○ B. Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values"
- ◉ C. Change fields that may not exist in source documents from required to optional, allowing the model to omit them
- ○ D. Add a "confidence" field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total—sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals.

What's the most effective approach to improve extraction reliability?

- ◉ A. Add a "calculated_total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.
- ○ B. Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.
- ○ C. Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining which extracted values are most likely correct.
- ○ D. Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies.

What's the most effective approach to improve extraction accuracy for documents with amendments?

- (●) A. Redesign the schema so amended fields capture multiple values, each with source location and effective date.
- ( ) B. Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.
- ( ) C. Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.
- ( ) D. Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Monitoring shows 12% of extractions fail Pydantic validation with specific errors like "expected float for quantity, got '2 to 3'". Retrying these requests without modification produces identical failures.

What's the most effective approach to recover from these validation failures?

- ( ) A. Set temperature to 0 to eliminate output variability and ensure consistent formatting
- ( ) B. Implement a secondary pipeline using a larger model tier to reprocess documents that fail validation
- (●) C. Send a follow-up request including the validation error, asking the model to correct its output
- ( ) D. Pre-process source documents to standardize problematic formats before sending them for extraction

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures.

What is the most reliable approach to ensure Claude's output consistently matches the schema?

- (●) A. Define a tool with an input schema matching your required JSON structure and extract the data from Claude's tool_use response.
- ( ) B. Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses.
- ( ) C. Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.
- ( ) D. Make two separate API calls—first extracting information as text, then asking Claude to format that text as JSON.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for attendee_count when the source contains no attendance information.

What's the most effective way to reduce these false extractions?

- ○ A. Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.
- ○ B. Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.
- ○ C. Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.
- ◉ D. Add prompt instructions to return null for any field where information is not directly stated in the source.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system has been running for 3 weeks and human reviewers have corrected 847 extractions. Analysis reveals a recurring pattern: when recipes use informal measurements like "a handful" or "a splash," the model either invents specific amounts or leaves fields empty—accounting for 23% of all corrections.

How should you use this feedback to improve extraction accuracy?

- ◉ A. Update your JSON schema to add a "measurement_type" enum field (precise/informal).
- ○ B. Add few-shot examples to your prompt demonstrating correct handling of informal measurements—extracting them verbatim rather than converting or omitting them.
- ○ C. Fine-tune the model on the 847 corrected extractions.
- ○ D. Implement a post-processing layer that uses pattern matching to detect informal measurement phrases in source text and automatically populate values when the extraction is empty.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned."

What's the most effective improvement?

- ○ A. Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.
- ○ B. Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.
- ◉ C. Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- ○ D. Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence ≥90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions.

Before deploying, what validation step is most critical?

- (●) A. Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.
- ( ) B. Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.
- ( ) C. Run a two-week pilot routing 25% of high-confidence extractions directly to downstream systems and monitor error reports.
- ( ) D. Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your system must extract event details from calendar invitations and output JSON that strictly conforms to a schema with fields for title, date, time, location, and attendees. Downstream systems reject any malformed or non-conformant JSON.

What approach provides the most reliable schema compliance?

- ( ) A. Include detailed JSON formatting instructions and the target schema in your prompt, then parse Claude's text response as JSON.
- ( ) B. Append instructions like "Output only valid JSON matching the schema exactly" and implement retry logic to re-prompt when JSON parsing fails.
- (●) C. Define a tool with your target schema as input parameters and have Claude call it with the extracted data.
- ( ) D. Pre-fill Claude's response with an opening brace to force JSON output, then complete and parse the response.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

The system routes documents with extraction confidence below 85% to human review. A quarterly audit reveals that 12% of high-confidence extractions (≥85%) also contain errors—cases where the model finds plausible-but-incorrect values. Error sources vary: comparison tables showing competitor specs, appendices referencing different product variants, and ambiguous phrasing the model misinterprets. You need a sustainable strategy to catch these high-confidence errors and measure whether improvements reduce the error rate over time.

What approach is most effective?

- (●) A. Implement stratified random sampling reviewing a fixed percentage of high-confidence extractions weekly, enabling error rate measurement and novel pattern detection.
- ( ) B. Add a verification pass that re-extracts from each high-confidence document, flagging cases where the two extraction attempts produce different results.
- ( ) C. Implement heuristic rules that flag documents containing comparison tables or appendices for review regardless of confidence score.
- ( ) D. Lower the confidence threshold from 85% to 70%, routing a larger volume of extractions to human review.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction pipeline validates outputs against JSON schemas, but you need to implement human review given limited reviewer capacity (they can handle approximately 5% of total extraction volume).

What's the most effective basis for selecting which extractions to route for human review?

- A. Randomly sample 5% of extractions for review.
- B. Route extractions containing specific high-priority entity types (e.g., financial figures, dates) for human review, regardless of extraction confidence.
- C. Route extractions for review only when downstream systems report data quality issues or processing failures.
- D. Route extractions where the model indicates low confidence or where source documents contain ambiguous or contradictory information.

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JavaScript Object Notation (JSON) schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Your extraction system implements automatic retries when validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts.

For which failure pattern would additional retries be LEAST effective?

- A. The model extracts dates as ISO 8601 datetime strings ("2023-03-15T00:00:00Z") when the schema requires only the date portion (YYYY-MM-DD)
- B. The model extracts citation counts as locale-formatted strings ("1,234") when the schema requires integers
- C. The model extracts "et al." for co-authors when the full list exists only in an external document not in the input
- D. The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

After expanding the agent's MCP tools with delivery-specific capabilities (`check_delivery_status`, `contact_driver`, `issue_credit`, `apply_promo_code`, `update_delivery_address`, `reschedule_delivery`), the total tool count has grown from 4 to 10. Your evaluation suite shows tool selection accuracy has dropped from 88% to 71%. Log analysis reveals the majority of errors involve the agent selecting between semantically overlapping tools — calling `issue_credit` when `process_refund` was correct, and calling `check_delivery_status` when `lookup_order` already returns the needed data.

Which approach structurally eliminates the semantic overlap identified in the logs as the error source?

- A. Consolidate semantically overlapping tools — merge `issue_credit` and `process_refund` into a single `resolve_compensation` tool with an `action` parameter, and fold `check_delivery_status` into `lookup_order` with an optional `include_tracking` flag.
- B. Add few-shot examples to the system prompt demonstrating correct selection for each ambiguous tool pair, such as showing when `issue_credit` applies versus when `process_refund` is appropriate.
- C. Enable the tool search tool with `defer_loading` on the six new tools, keeping the original four always loaded, so the agent dynamically discovers specialized tools only when needed.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

You're implementing the escalation logic for when the agent should call `escalate_to_human`. Your team proposes four different approaches for triggering escalation.

Which approach will most reliably identify cases that genuinely require human intervention?

- (●) A. Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.
- ( ) B. Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.
- ( ) C. Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.
- ( ) D. Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Compliance requires that refunds exceeding $500 must automatically escalate to a human agent—this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate).

How should you achieve guaranteed compliance?

- ( ) A. Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over $500 MUST trigger human escalation. NEVER process these directly."
- ( ) B. Modify the refund tool to return an error with message "Amount exceeds policy limit—please escalate" when threshold is exceeded.
- ( ) C. Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts ($400, $500, $600).
- (●) D. Implement a hook to intercept tool calls; when the refund process amount exceeds $500, block it and invoke human escalation.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your `process_refund` tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (~5% of calls), and business errors ("Order exceeds 30-day return window", "Item already refunded") that are permanent (~12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude.

What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- ( ) A. Add a `check_refund_eligibility` tool that must be called before `process_refund` to prevent business rule violations.
- ( ) B. Add few-shot examples showing how to distinguish retriable from non-retriable errors by parsing error message text.
- (●) C. Return structured error responses with `retriable: false` for business errors and a customer-friendly explanation for Claude to use.
- ( ) D. Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened.

What's the most likely cause of this behavior?

- A. Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.
- B. The prompt lacks instructions telling Claude to remember information across multiple exchanges.
- C. The conversation history isn't being passed in subsequent API requests.
- D. The verification tool is clearing the agent's internal state after each successful validation step.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer contacts the agent about a warranty claim on a power drill. Resolving this requires multiple sequential tool calls: `get_customer` to look up their account, `lookup_order` to find the purchase details, and then either `process_refund` or `escalate_to_human` depending on warranty eligibility. You're implementing the agentic loop that orchestrates these steps using the Claude API.

What is the primary mechanism your application uses to determine whether to continue the loop or stop?

- A. You track the number of tool calls made and exit the loop once a preconfigured maximum is reached.
- B. You manually set the `tool_choice` parameter to `"none"` after the final expected tool call to force Claude to stop requesting tools.
- C. You check the `stop_reason` field in each API response — the loop continues while it equals `"tool_use"` and exits when it changes to `"end_turn"` or another terminal value.
- D. You check whether Claude's response contains a text content block — if text is present, the agent has produced its final answer and the loop should exit.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production logs show that when the agent handles complex billing disputes requiring 6+ tool calls, it sometimes exhausts its `max_turns` limit after gathering data but before completing resolution or escalating. The team's goal is to guarantee that every customer interaction ends with either a completed resolution or a human handoff, regardless of how the agent loop terminates.

Which approach achieves this guarantee?

- A. Split the workflow into two sequential agent invocations — a first agent gathers information via `get_customer` and `lookup_order`, then a second agent receives that data and handles `process_refund` or `escalate_to_human`, each with separate turn budgets.
- B. Add orchestration-layer code that checks the agent's outcome after each loop termination — if the loop ended without a completed resolution or escalation, programmatically call `escalate_to_human` with the accumulated conversation context and tool results.
- C. Implement a pre-tool-use hook that counts tool invocations and terminates the loop with an automatic escalation once the agent reaches 80% of its `max_turns` limit.
- D. Add system prompt instructions telling the agent to call `escalate_to_human` with a summary of its findings whenever it determines it cannot complete resolution within its remaining actions.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

During testing, you find that when a customer says "I need a refund for my recent purchase," the agent calls `process_refund` immediately — but populates the required `order_id` parameter with a plausible-looking but fabricated value instead of first calling `lookup_order` to retrieve the actual order ID. The refund call fails because the fabricated ID doesn't exist.

Which change directly addresses the root cause of the agent fabricating the `order_id` value?

- ○ A. Add server-side validation that checks whether the `order_id` exists in your database before executing the refund, returning an error to the agent if not found.
- ○ B. Switch `tool_choice` from `"auto"` to `"any"` to force the agent to make a tool call on every turn.
- ◉ C. Update the `process_refund` tool description to explicitly state that `order_id` must be obtained from a prior `lookup_order` call and must never be assumed or invented.
- ○ D. Pre-parse incoming customer messages to extract any order IDs mentioned, and inject them into the conversation context before passing to Claude.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains `lookup_order` results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information.

What approach most reliably handles returning customers?

- ○ A. Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.
- ○ B. Resume with full history but filter out previous tool_result messages before resuming, keeping only the human/assistant turns so the agent must re-fetch needed data.
- ○ C. Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.
- ◉ D. Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Production logs reveal inconsistent error handling: when `lookup_order` fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: `{"isError": true, "content": [{"type": "text", "text": "Operation failed"}]}`. The agent cannot distinguish between error types.

What's the most effective improvement?

- ○ A. Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.
- ◉ B. Enhance error responses with structured metadata: include `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure.
- ○ C. Create an `analyze_error` MCP tool the agent calls after any failure to determine the error category and recommended action.
- ○ D. Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

When the agent calls `lookup_order` and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call `process_refund` or `escalate_to_human` next?

- ◉ A. The order details are added to the conversation and the model reasons about which action to take.
- ○ B. The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.
- ○ C. The orchestration layer automatically routes to the next tool based on the order's status field.
- ○ D. The agent executes the remaining steps in a tool sequence planned at the start of the request.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

During a billing dispute resolution, your agent successfully retrieves customer info via `get_customer` and order details via `lookup_order`, but when attempting to call `process_refund`, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure.

What approach best balances first-contact resolution with appropriate error handling?

- ○ A. Escalate immediately to a human agent since the refund action cannot be completed
- ◉ B. Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later
- ○ C. Implement automatic retries with exponential backoff for `process_refund`, keeping the conversation open until the refund is successfully processed
- ○ D. Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

A customer raises three separate issues during one session: a refund inquiry (turns 1-15), a subscription question (turns 16-30), and a payment method update (turns 31-45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits.

What strategy best maintains the agent's ability to address all issues throughout the session?

- A. Summarize earlier turns into a narrative description, preserving full message history only for the active issue.
- B. Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.
- C. Implement sliding window context that retains the most recent 30 turns.
- D. Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Anthropic's tool use documentation states: "Write instructive error messages. Instead of generic errors like 'failed', include what went wrong and what Claude should try next." A billing dispute agent uses `lookup_order`, which catches all exceptions and returns a `tool_result` with `is_error: true` and the message `"Tool execution failed"`. Monitoring shows two failure modes: the agent retries the identical call until hitting the turn limit, or it immediately calls `escalate_to_human` without trying alternative tools.

Which change follows the documented recommendation and gives Claude the information it needs to select the correct recovery action for each error type?

- A. Remove `is_error: true` and return the error details as normal tool content, so Claude reasons about the response as data rather than treating it as a flagged failure condition that biases retry behavior.
- B. Return error-type-specific messages with `is_error: true`, e.g., `"Order not found—try get_customer to search by phone"` for data errors and `"Database timeout (transient)—retry should succeed"` for infrastructure errors.
- C. Add an error classification step in the agentic loop that intercepts tool errors before Claude sees them, tags each as "retry," "try_alternative," or "escalate," and appends that recommendation to the tool result.
- D. Implement retry logic with exponential backoff inside each tool implementation so transient errors are

You are building a customer support resolution agent using the Claude Agent SDK. The agent handles high-ambiguity requests like returns, billing disputes, and account issues. It has access to your backend systems through custom Model Context Protocol (MCP) tools (get_customer, lookup_order, process_refund, escalate_to_human). Your target is 80%+ first-contact resolution while knowing when to escalate.

Your agent is handling a billing dispute. After calling `get_customer` and `lookup_order`, it identifies that the dispute involves a promotional pricing error requiring manager approval—beyond the agent's authorization level.

How should the workflow handle this mid-process escalation?

- A. Persist the complete conversation and tool response history to a database, then call `escalate_to_human` with a reference ID.
- B. Call `escalate_to_human` passing only the customer's original message.
- C. Attempt the refund with `process_refund` anyway, escalating only if the system rejects the transaction.
- D. Compile a structured handoff with customer details, order info, and the identified issue before calling `escalate_to_human`.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team has connected a custom MCP server that provides DevOps workflow templates. The server exposes several MCP prompts (such as `deploy_checklist` and `incident_response`) in addition to tools.

How do these MCP prompts become accessible within Claude Code?

- ○ A. They are added to Claude Code's tool registry alongside the server's tools, invoked automatically by the model when relevant to the task.
- ○ B. They are surfaced as `@`-mentionable resources alongside files, fetched and attached to your message when referenced.
- ○ C. They are automatically prepended to every conversation as additional system-level context, influencing Claude's behavior throughout the session.
- ◉ D. They appear as slash commands (e.g., `/mcp__servername__deploy_checklist`) that you can invoke, with arguments passed after the command name.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a complex graph traversal algorithm with specific performance requirements and edge cases to handle (disconnected nodes, cycles, weighted edges). You want to structure your workflow for efficient iterative refinement with Claude.

What approach will most effectively enable progressive improvement across multiple iterations?

- ○ A. Provide Claude with a detailed natural language specification of the algorithm, including all requirements and edge cases. Review each output manually and provide descriptive feedback on what behavior needs to change.
- ◉ B. Write a test suite covering expected behavior, edge cases, and performance requirements before implementation. Ask Claude to write code that passes the tests, then iterate by sharing test failures with each refinement request.
- ○ C. Have Claude extensively research the algorithm and create a detailed implementation plan using extended thinking, then implement the complete solution based on that plan.
- ○ D. Provide Claude with a reference implementation from documentation, then ask it to rewrite the code to match your codebase style and add the required edge case handling, comparing outputs against the reference.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team frequently migrates React components to Vue. You've written a step-by-step workflow for Claude Code to follow during each migration, and you want every developer on the team to invoke it by typing /migrate-component. The workflow should stay in sync as the team iterates on it.

Where should you place the skill file?

- ○ A. In ~/.claude/skills/migrate-component/SKILL.md on each developer's machine
- ◉ B. In .claude/skills/migrate-component/SKILL.md at the project root, committed to version control
- ○ C. In the project's .claude/settings.json using a skillOverrides entry to register and define the workflow
- ○ D. As a detailed instruction block in the project's root CLAUDE.md file

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You need to add a date validation check ensuring event dates are in the future. This requires adding a conditional statement to one existing function in a single file.

What is the most appropriate approach?

- A. Start with extended thinking mode enabled to ensure thorough reasoning about the validation logic
- B. Enter plan mode first to create a detailed implementation strategy before making the change
- C. Use direct execution to make the change
- D. Enter plan mode to analyze how the validation might impact other parts of the reservation flow

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a new payment processing module that must follow your project's established patterns for database transactions, error handling, and audit logging. You've identified three existing modules that exemplify these patterns: `db_utils.py`, `error_handlers.py`, and `audit_logger.py`. This is a one-off integration task—these patterns are well-documented in your team wiki and don't need additional project-level documentation.

What's the most effective approach?

- A. Describe the patterns from the three modules in natural language in your prompt, explaining the transaction handling approach, error format, and logging conventions Claude should follow.
- B. Ask Claude to explore your codebase to find and understand the transaction, error handling, and logging patterns before generating the new module.
- C. Use @ references to include the three modules directly in your prompt, giving Claude concrete code examples of the patterns to follow.
- D. Add documentation of each pattern to your CLAUDE.md file, establishing them as project conventions that Claude will apply automatically.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've asked Claude to write a data migration script, but the initial output doesn't correctly handle records with null values in required fields.

What's the most effective way to iterate toward a working solution?

- A. Provide a test case with example input containing null values and the expected output, then ask Claude to fix it.
- B. Add "think harder about edge cases" to your prompt and request a complete rewrite of the migration logic.
- C. Manually edit the generated code to fix the null handling, then continue working with Claude on other parts.
- D. Describe the null value problem in detail and ask Claude to regenerate the entire script with improved edge case handling.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team wants Claude to follow a detailed code review checklist (8 items covering API changes, test coverage, documentation, security, etc.) when reviewing pull requests. The team also uses Claude extensively for other tasks: writing new features, debugging production issues, and generating documentation. Currently, developers paste the checklist at the start of each review session.

Which approach best addresses this workflow need?

- A. Add the checklist to the project's CLAUDE.md file under a "Code Review" section.
- B. Create a `/review` slash command containing the checklist, invoked when starting reviews.
- C. Configure plan mode as the default for code review sessions.
- D. Create a dedicated review subagent with the checklist embedded in its configuration.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team has three requirements for Claude Code's behavior in your project: 1. Claude must never modify files in the `db/migrations/` directory 2. Claude should prefer your custom logging module over `console.log` 3. All TypeScript files must be auto-formatted with Prettier after every edit All three are currently written as instructions in your project's CLAUDE.md. During a complex refactoring session, a developer discovers that Claude edited a migration file, violating requirement #1.

How should you restructure these requirements across Claude Code's configuration mechanisms?

- A. Move all three requirements into `.claude/rules/` as path-scoped rules: one targeting `db/migrations/**` that forbids editing those files, and others targeting `**/*.ts` for the logging convention and formatting instruction.
- B. Add `Edit(./db/migrations/**)` to `permissions.deny` in the project settings, keep the logging preference in CLAUDE.md, and add a `PostToolUse` hook on the Edit tool that runs Prettier on changed TypeScript files.
- C. Configure hooks for all three: a `PreToolUse` hook script that blocks Edit calls targeting `db/migrations/`, a `PreToolUse` hook script that adds logging convention context before edits, and a `PostToolUse` hook that runs Prettier after TypeScript edits.
- D. Rewrite all three requirements in CLAUDE.md using stronger directive language and add few-shot examples that demonstrate Claude refusing to edit migration files and running Prettier after edits.

21
08.2026_15.27.21_REC

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team is configuring MCP servers in Claude Code. You want to add a shared venue lookup server that all team members should have access to, and you personally want to add an experimental music playlist server that only you are testing.

Which configuration approach correctly applies MCP server scopes?

- A. Add venue server to `~/.claude.json` and playlist server to `.mcp.json`
- B. Add both servers to the project-level `.mcp.json` file
- C. Add venue server to `.mcp.json` and playlist server to `~/.claude.json`
- D. Add both servers to your local `~/.claude.json`

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your team's CLAUDE.md includes a rule: "Use 4-space indentation and always run Prettier formatting." Despite this, code reviews reveal that roughly 30% of files Claude Code generates use inconsistent formatting — sometimes 2-space indentation, sometimes missing trailing commas. Adding emphasis ("IMPORTANT: You MUST use Prettier formatting") reduces violations to about 15%, but doesn't eliminate them.

What is the most effective way to ensure all generated code is consistently formatted?

- A. Add a Stop hook with a prompt-based check that evaluates whether generated code follows formatting standards and prompts Claude to fix violations.
- B. Extract the formatting rules into a dedicated skill that Claude loads automatically when generating code, with more detailed examples of correct formatting.
- C. Configure a PostToolUse hook with an Edit|Write matcher that automatically runs Prettier on each file Claude modifies.
- D. Split the formatting rules into path-scoped .claude/rules/ files that load when Claude works on matching file types.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

A security audit requires updating your authentication library from v2 to v3. The migration guide documents breaking changes: `authenticate()` now returns a Promise instead of accepting a callback, the `User` type has restructured fields, and three deprecated methods were removed. Grep shows the library is imported in 45 files across several modules.

What's the most effective approach?

- A. Paste the migration guide's breaking changes into your prompt and use direct execution to update all usages across the 45 files.
- B. Update the dependency version, run the test suite, and use Claude Code to fix each failure as it appears.
- C. Create a custom slash command encapsulating the migration transformations, then execute it against each file without prior codebase exploration.
- D. Enter plan mode to explore library usage across modules, map affected code paths, then create a migration strategy before implementing.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're tasked with adding real-time updates to the application. This could be implemented using WebSockets, Server-Sent Events, or polling, each with different complexity, browser support, and infrastructure requirements.

What's the most effective way to begin this task?

- ○ A. Use direct execution with a prompt asking Claude to analyze all approaches and implement the one it determines is best.
- ○ B. Enter plan mode to explore the architecture, evaluate trade-offs, and present options for team approval before implementing.
- ◉ C. Use direct execution to implement polling first, then evaluate whether to upgrade to WebSockets later.
- ○ D. Start direct execution with WebSockets, then refactor if infrastructure issues arise.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You're implementing a caching layer for API responses to speed up the `/products` endpoint. You have a rough idea—Redis with a 5-minute TTL—but you're new to production caching and aren't sure what other considerations a robust implementation requires.

What's the most effective way to start your iterative workflow?

- ○ A. Write a specification with your known requirements and "TBD" markers for uncertain areas, having Claude propose solutions for each TBD as it implements.
- ○ B. Start with a minimal request: "Add Redis caching to /products with 5-minute TTL." Add features and fix issues through follow-up prompts as problems surface during testing.
- ○ C. Ask Claude to interview you about the caching requirements before implementing, surfacing considerations like invalidation strategies, cache layers, consistency guarantees, and failure modes.
- ◉ D. Use plan mode to analyze the current /products endpoint implementation, then provide your caching requirements once Claude explains how the existing code is structured.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

Your infrastructure-as-code repository includes Terraform modules (`/terraform/`), Kubernetes manifests (`/kubernetes/`), and CI/CD pipeline scripts (`/pipelines/`). Each requires different conventions, but your single root CLAUDE.md has grown to 500+ lines. When developers work on Kubernetes files, Terraform-specific rules load into context unnecessarily, consuming tokens.

What is the best approach to reorganize so only relevant guidance loads when editing specific file types?

- ○ A. Split content into subdirectory CLAUDE.md files (`/terraform/CLAUDE.md`, `/kubernetes/CLAUDE.md`), so Claude loads directory-specific guidance.
- ○ B. Keep the root CLAUDE.md and use `@path/to/import` syntax to modularly include tool-specific guidance files from separate documents.
- ○ C. Restructure the root CLAUDE.md into clearly labeled sections with headers (e.g., "## Terraform Conventions"), improving organization and readability.
- ◉ D. Create files in `.claude/rules/` with YAML frontmatter path-scoping (e.g., `paths: ["terraform/**/*"]`), loading rules only when editing matching files.

You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution.

You've asked Claude Code to build a PDF report generation feature. The initial implementation queries the database correctly, but the output has formatting issues: table columns are too narrow causing content truncation, dates display without proper formatting, and page break handling is incorrect. You've noticed these issues interact—changing column widths affects how dates render, and page breaks depend on content height.

What's the most effective approach for iterating toward a working solution?

- (●) A. Address the column width issue first with specific measurements, verify it works, then fix date formatting within the corrected columns, then adjust page breaks—testing after each change.
- ( ) B. Start fresh with a detailed prompt specifying all formatting requirements upfront.
- ( ) C. Show Claude an example of a correctly formatted report and ask it to match that output, rather than listing the specific technical issues.
- ( ) D. Provide all three issues in a single detailed message with exact specifications for each, allowing Claude to address them together in one update.